ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống hỏi đáp cho sinh viên Đại học Bách Khoa Hà Nội

NGÔ THÀNH NAM
nam.nt194127@sis.hust.edu.vn

Ngành: Khoa học máy tính

Giảng viên hướng dẫn: TS. Ngô Văn Linh _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2024

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời tri ân sâu sắc nhất tới thầy TS. Ngô Văn Linh, người đã tận tình giúp đỡ và chỉ dẫn em trong suốt quá trình thực hiện đồ án tốt nghiệp này. Những lời khuyên quý báu và sự hỗ trợ nhiệt tình của thầy đã giúp em vượt qua nhiều khó khăn, hoàn thành tốt đồ án của mình.

Em cũng xin bày tỏ lòng biết ơn chân thành tới các thầy cô của Trường Công nghệ Thông tin và Truyền thông - Đại học Bách Khoa Hà Nội. Chính nhờ những kiến thức quý báu và sự hướng dẫn tận tình của các thầy cô mà em đã có nền tảng vững chắc để hoàn thành đồ án này cũng như là trên con đường phát triển sự nghiệp của mình. Cảm ơn Đại học Bách Khoa Hà Nội đã tạo cho em một môi trường học tập tuyệt vời, nơi em có cơ hội gặp gỡ những người bạn cùng chí hướng và cùng nhau trải qua những kỷ niệm đáng nhớ của thời sinh viên.

Con xin gửi lời cảm ơn chân thành tới ông bà, bố mẹ và em trai, những người luôn ủng hộ và động viên con trong mọi bước đi. Tình yêu thương và sự quan tâm của gia đình chính là động lực to lớn giúp con vượt qua mọi thử thách để đạt được thành công ngày hôm nay.

Và cuối cùng, em xin tự gửi lời cảm ơn tới bản thân mình vì đã luôn kiên trì, quyết tâm và nỗ lực hết mình để hoàn thành đồ án tốt nghiệp này. Đây là một hành trình đầy thử thách nhưng cũng rất đáng nhớ và tự hào !

LỜI CAM KẾT

Họ và tên sinh viên: Ngô Thành Nam
MSSV: 20194127
Điện thoại liên lạc: 0376033291
Email: nam.nt194127@sis.hust.edu.vn
Lớp: Khoa học máy tính 05-K64
Chương trình đào tạo: Khoa học máy tính

Tôi – *Ngô Thành Nam* – cam kết Đồ án Tốt nghiệp (ĐATN) là công trình nghiên cứu của bản thân tôi dưới sự hướng dẫn của *Tiến sĩ Ngô Văn Linh*. Các kết quả nêu trong ĐATN là trung thực, là thành quả của riêng tôi, không sao chép theo bất kỳ công trình nào khác. Tất cả những tham khảo trong ĐATN – bao gồm hình ảnh, bảng biểu, số liệu, và các câu từ trích dẫn – đều được ghi rõ ràng và đầy đủ nguồn gốc trong danh mục tài liệu tham khảo. Tôi xin hoàn toàn chịu trách nhiệm với dù chỉ một sao chép vi phạm quy chế của nhà trường.

Hà Nội, ngày 30 tháng 06 năm 2024

Tác giả ĐATN

Ngô Thành Nam

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án tốt nghiệp của em tập trung vào việc xây dựng hệ thống hỏi đáp cho sinh viên Đại học Bách Khoa Hà Nội, đáp ứng nhu cầu cấp thiết khi hàng ngày có rất nhiều câu hỏi được đặt ra trên diễn đàn hỏi đáp của trường. Các câu hỏi này thường đa dạng về nhiều chủ đề khác nhau, vậy nên việc tìm kiếm thông tin cũng như trả lời các câu hỏi vẫn còn là một thách thức lớn. Hiện tại, chưa có hệ thống nào hỗ trợ đáp ứng vấn đề này một cách hiệu quả.

Về hướng tiếp cận bài toán, em phát triển hệ thống của mình thành hai mô-đun chính đó là (i) mô-đun truy xuất và (ii) mô-đun sinh câu trả lời. Đối với mô-đun truy xuất em kết hợp nhiều mô hình truy xuất khác nhau từ những mô hình dựa trên so khớp từ vựng cho đến những mô hình học biểu diễn ngữ nghĩa sâu của văn bản để tận dụng được thế mạnh của từng mô hình. Nhiều kiến trúc mô hình mới cho tác vụ truy xuất thông tin cũng được em sử dụng để có được kết quả tốt nhất cho mô-đun này. Đối với mô-đun sinh câu trả lời, em tận dụng sức mạnh suy luận của các mô hình ngôn ngữ lớn mã nguồn mở để sinh ra câu trả lời chính xác và tự nhiên dựa trên phần thông tin được trích ra từ mô-đun truy xuất.

Với mô-đun truy xuất, ban đầu mô hình Transformer Encode sẽ được em mang đi tiền huấn luyện trên tập dữ liệu thô và sau khi đã tiền huấn luyện xong sẽ sử dụng để tinh chỉnh cho các mô hình bi-encoder và cross-enocder. Dữ liệu dùng để tinh chỉnh cho các mô hình bi-encoder được lấy từ việc kết hợp kỹ thuật chia đoạn (chunking) dữ liệu và khai thác các mẫu âm khó. Sự kết hợp này sẽ vừa tăng chất lượng của việc học biểu diễn tương phản và đồng thời khắc phục các trường hợp mà văn bản cần truy xuất bị quá dài. Tập dữ liệu này cũng được em lựa chọn để tinh chỉnh mô hình M3-Embedding và với việc sử dụng kỹ thuật tự chắt lọc tri thức (Self-Knowledge Distillation) giúp mô hình M3-Embedding có thể huấn luyện đồng thời ba chức năng truy xuất bao gồm truy xuất dày đặc (Dense Retrieval), truy xuất thưa thớt (Sparse Retrieval) và truy xuất đa véc-tơ (Multi-vector Retrieval) từ đó có thể nâng cao chất lượng của từng loại chức năng truy xuất này. Sau cùng cả ba chức năng truy xuất của mô hình M3-Embedding cùng với các mô hình bi-encoder được em tổng hợp lại để tận dụng được thế mạnh của từng mô hình (ensemble learning). Bên cạnh đó em cũng thực hiện việc xếp hạng lại sử dụng mô hình cross-encoder để tận dụng được sự hiệu quả về độ chính xác cao của mô hình này và em cũng bổ trợ thêm kết quả của mô hình BM25 vào mô-đun truy xuất để có thể tận dụng được khả năng so khớp từ vựng của mô hình này.

Với mô-đun sinh câu trả lời, em lựa chọn sử dụng kỹ thuật prompting để bổ sung

ngữ cảnh cho mô hình ngôn ngữ lớn dựa trên câu truy vấn và các đoạn thông tin phù hợp đã trích xuất ra được từ mô-đun truy xuất. Sau đó nhờ vào sức mạnh suy luận của mô hình này để sinh ra câu trả lời cuối cùng.

Đóng góp chính của em trong đồ án tốt nghiệp này là việc xây dựng một hệ thống hỏi đáp hoàn chỉnh, có thể hỗ trợ giải đáp các câu hỏi của các bạn sinh viên trong quá trình học tập tại trường. Kết quả em đạt được sau đồ án là hệ thống đạt được độ chính xác cao trên tổng thể cả 2 mô-đun truy xuất và sinh câu trả lời, đồng thời hệ thống cũng đáp ứng được yêu cầu về thời gian khi áp dụng trong thực tiễn.

ABSTRACT

My graduation thesis focuses on constructing a Question-Answering system for undergraduates at Hanoi University of Science and Technology, addressing the critical need arising from the daily influx of questions on the university's Q&A platform. These questions cover a wide range of topics, making information retrieval and providing answers still significant challenges. Currently, there is no system that effectively meets this demand.

In approaching the problem, I have divided my system into two main modules: (i) retrieval and (ii) answer generation. For the retrieval module, I integrated various retrieval models ranging from vocabulary-based matching models to deep semantic text representation models to leverage the strengths of each. I also employed several novel model architectures for information retrieval tasks to achieve optimal results for this module. Regarding the answer generation module, I utilized the inferential power of large open-source language models to generate accurate and natural answers based on the information extracted from the retrieval module.

For the retrieval module, I initially pre-trained the Transformer Encoder model on raw data. Upon completing the pre-training, I fine-tuned it for both bi-encoder and cross-encoder models. The data used for fine-tuning the bi-encoder models was obtained through data chunking techniques and mining difficult samples. This approach enhances the quality of contrastive representation learning and addresses cases where the retrieved text is excessively lengthy. The same dataset was also selected for fine-tuning the M3-Embedding model, utilizing the Self-Knowledge Distillation technique. This technique simultaneously trained the M3-Embedding model for three retrieval functions: Dense Retrieval, Sparse Retrieval, and Multi-vector Retrieval, thereby enhancing the quality of each retrieval function. Lastly, I integrated all three retrieval functions of the M3-Embedding model along with the bi-encoder models to leverage the strengths of each model through ensemble learning. Additionally, I implemented re-ranking using the cross-encoder model to capitalize on its high accuracy efficiency and I also supplemented the results of the BM25 model into the retrieval module to leverage its vocabulary-matching capabilities.

For the answer generation module, I chose to use prompting techniques to provide context to the large language model based on the query and relevant passages extracted from the retrieval module. Subsequently, I relied on the inferential power of this model to generate the final answer.

My main contribution to this thesis is developing a complete Question-Answer

system that can assist in answering questions posed by undergraduates during their studies. The outcome achieved from this project is a system that demonstrates high accuracy across both modules: retrieval and answer generation. Additionally, the system meets the practical requirements regarding response time when implemented in real-world scenarios.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1

1.1 Đặt vấn đề 1

1.2 Các giải pháp hiện tại và hạn chế 1

1.3 Mục tiêu và định hướng giải pháp 3

1.4 Phạm vi của đồ án 4

1.5 Đóng góp của đồ án 4

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT 6

2.1 Các loại chức năng truy xuất 6

2.2 Các thuật toán truy xuất thưa thớt 7

2.2.1 TF-IDF 7

2.2.2 BM25 8

2.2.3 Ưu và nhược điểm của TF-IDF và BM25 9

2.3 Học tương phản trong truy xuất thông tin 9

2.3.1 Nguyên lý của học tương phản 9

2.3.2 Các kỹ thuật thường được sử dụng trong học tương phản 10

2.4 Hướng tiếp cận Bi-encoder 10

2.4.1 Tiền huấn luyện Masked Language Model 11

2.4.2 Condenser và phương pháp cải tiến 12

2.5 Hướng tiếp cận Cross-encoder 16

2.6 M3-Embedding 17

2.6.1 Truy xuất kết hợp (Hybrid retrieval) 18

2.6.2 Self-Knowledge Distillation 19

2.6.3 Huấn luyện mô hình M3-Embedding 20

2.7 Phương pháp Ensemble 20

2.8 Kỹ thuật prompting 21

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT 23

3.1 Tổng quan giải pháp 23

3.1.1 Backbone sử dụng 23

3.1.2 Tập dữ liệu 24

3.1.3 Các thành phần của mô-đun truy xuất 26

3.1.4 Kết hợp với BM25 27

3.2 Thành phần Retriever 27

3.2.1 Quá trình tiền huấn luyện 28

3.2.2 Quá trình tinh chỉnh 28

3.3 Thành phần Re-ranker 29

3.4 Mô-đun sinh câu trả lời 30

CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM 32

4.1 Phương pháp đánh giá 32

4.1.1 Các tham số đánh giá 32

4.1.2 Các tiêu chí đánh giá 32

4.2 Thử nghiệm 32

4.2.1 Các baseline sử dụng để so sánh 32

4.2.2 Cấu hình các tham số 33

4.2.3 Các phương án thử nghiệm 33

4.3 Thử nghiệm tinh chỉnh mô hình bi-encoder với từng hàm mất mát khác nhau 35

4.4 So sánh kết quả khi tinh chỉnh pha 1 với backbone chưa được tiền huấn luyện và đã được tiền huấn luyện thêm 36

4.5 Tinh chỉnh mô hình bi-encoder với backbone được tiền huấn luyện với các phương pháp khác nhau 37

4.6 So sánh kết quả khi tinh chỉnh các mô hình bi-encoder pha 1 với dữ liệu trước và sau khi chunking 38

4.7 So sánh kết quả khi tinh chỉnh mô hình bi-encoder pha 1 và pha 2 với dữ liệu sau khi chunking..... 39

4.8 So sánh kết quả khi tinh chỉnh mô hình M3-Embedding với các mô hình bi-encoder với cùng dữ liệu pha 1 39

4.9 So sánh kết quả khi tinh chỉnh mô hình M3-Embedding với dữ liệu pha 1 và pha 2 40

4.10 So sánh kết quả với các mô hình baseline 42

4.11 Kết quả trên toàn bộ mô-đun truy xuất..... 43

4.12 So sánh về thời gian truy xuất các mô hình trong mô-đun truy xuất 43

4.13 Kết quả kiểm thử toàn hệ thống..... 47

CHƯƠNG 5. KẾT LUẬN 51

5.1 Kết luận..... 51

5.2 Hướng phát triển tiếp theo 52

DANH MỤC HÌNH VẼ

Hình 2.1 Kiến trúc bi-encoder 11
Hình 2.2 Ví dụ về Masked Language Model 13
Hình 2.3 Kiến trúc Condenser 14
Hình 2.4 Kiến trúc cross-encoder 17
Hình 2.5 Quy trình huấn luyện của M3-Embedding với tự chắt lọc tri thức 20

Hình 3.1 Tổng quan hệ thống hỏi đáp 23
Hình 3.2 Phân bổ số lượng văn bản cho từng chủ đề 24
Hình 3.3 Luồng hoạt động của mô-đun truy xuất 27
Hình 3.4 Luồng huấn luyện các mô hình trong thành phần Retriever . . 28
Hình 3.5 Luồng hoạt động của mô-đun sinh câu trả lời 30

Hình 4.1 Minh họa khi tương tác với hệ thống - phần 1 47
Hình 4.2 Minh họa khi tương tác với hệ thống - phần 2 48
Hình 4.3 Minh họa khi tương tác với hệ thống - phần 3 49
Hình 4.4 Minh họa khi tương tác với hệ thống - phần 4 50

DANH MỤC BẢNG BIỂU

Bảng 3.1 Minh họa chi tiết về một nội dung trong một chủ đề 25
Bảng 3.2 Ví dụ về một mẫu dữ liệu trong tập dữ liệu truy xuất 26
Bảng 3.3 Phân bố độ dài trong tập dữ liệu văn bản 26
Bảng 3.4 Thống kê số cặp các bộ dữ liệu truy xuất pha 2 29

Bảng 4.1 Kết quả độ chính xác Acc@k (%) khi tinh chỉnh pha 1 mô hình PhoBERT-large bi-encoder với từng hàm mất mát 35
Bảng 4.2 Kết quả Recall@k (%) khi tinh chỉnh pha 1 mô hình PhoBERT-large bi-encoder với từng hàm mất mát 36
Bảng 4.3 Kết quả độ chính xác Acc(%) và Recall(%) khi tinh chỉnh mô hình bi-encoder với backbone chưa được tiền huấn luyện thêm và đã được tiền huấn luyện thêm 37
Bảng 4.4 Bảng kết quả độ chính xác Acc(%) và Recall(%) khi tinh chỉnh với backbone được tiền huấn luyện với các phương pháp khác nhau 38
Bảng 4.5 Kết quả độ chính xác Acc(%) và Recall(%) trước và sau khi chunking dữ liệu các mô hình bi-encoder pha 1 39
Bảng 4.6 Kết quả độ chính xác Acc(%) và Recall(%) của các mô hình bi-encoder pha 1 và pha 2 40
Bảng 4.7 Kết quả độ chính xác Acc(%) và Recall(%) của M3-Embedding so với các mô hình bi-encoder pha 1 41
Bảng 4.8 Kết quả độ chính xác Acc(%) và Recall(%) của mô hình M3-Embedding pha 1 và pha 2 41
Bảng 4.9 Kết quả độ chính xác Acc(%) và Recall(%) khi so sánh với các mô hình baseline 42
Bảng 4.10 Kết quả độ chính xác Acc(%) và Recall(%) trên toàn bộ mô-đun truy xuất 44
Bảng 4.11 Kết quả độ chính xác Acc(%) và Recall(%) trên toàn bộ mô-đun truy xuất 45
Bảng 4.12 Kết quả độ chính xác Acc(%) và Recall(%) trên toàn bộ mô-đun truy xuất 46
Bảng 4.13 So sánh thời gian truy xuất (đơn vị giây) của các mô hình trong mô đun truy xuất 46

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	**Ý nghĩa**
BERT	Bidirectional Encoder Representation from Transformer
CLS	Classification

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Tại môi trường đại học, việc am hiểu các quy định và quy chế đóng vai trò then chốt cho hành trình học tập hiệu quả của mỗi sinh viên. Tuy nhiên, với ma trận thông tin đồ sộ, không ít bạn sinh viên còn gặp khó khăn trong việc nắm bắt đầy đủ.

Bằng chứng là trên diễn đàn Engage - Mạng xã hội Đại học Bách Khoa Hà Nội, liên tục xuất hiện hàng loạt câu hỏi từ sinh viên xoay quanh các quy định thi cử, điểm rèn luyện, hoạt động trong ngoài nhà trường, thậm chí cả lĩnh vực bảo hiểm, y tế, gửi xe. Nhu cầu thông tin khổng lồ này đôi khi khiến diễn đàn quá tải, dẫn đến tình trạng trôi câu hỏi, ảnh hưởng đến công tác giải đáp và khiến sinh viên không nhận được phản hồi kịp thời. Hơn nữa, việc chỉ giải đáp thuần túy qua mạng không thể giải quyết triệt để mọi thắc mắc, bởi thực tế luôn tiềm ẩn nhiều trường hợp phức tạp. Mặc dù hầu hết thông tin đều được cung cấp đầy đủ trong các tài liệu và trang thông tin của nhà trường, nhưng do rải rác ở nhiều nơi, việc tìm kiếm gặp nhiều trở ngại.

Chính bởi vậy nên việc xây dựng một hệ thống hỏi đáp cho các bạn sinh viên Đại học Bách Khoa Hà Nội là vô cùng cần thiết. Hệ thống sẽ giúp cho các bạn sinh viên tìm kiếm thông tin nhanh chóng cũng như giúp cho các thầy cô trong việc trả lời câu hỏi. Hai mô-đun chính trong hệ thống là mô-đun truy xuất và mô-đun sinh câu trả lời trong đó: mô-đun truy xuất có vai trò lấy ra những văn bản phù hợp nhất với câu truy vấn trong khi mô-đun sinh câu trả lời sẽ dựa vào câu truy vấn và các văn bản được trích ra từ mô-đun truy xuất để tạo ra câu trả lời hoàn chỉnh.

1.2 Các giải pháp hiện tại và hạn chế

Các hệ thống hỏi đáp tự động đã được xuất hiện từ rất lâu với sự xuất hiện đầu tiên là các hệ thống dựa trên quy tắc (Rule-Based Systems). Các hệ thống dựa trên quy tắc này thì dễ triển khai trên miền kiến thức cụ thể tuy nhiên nó cũng có nhiều yếu điểm đó là phụ thuộc nhiều vào chuyên gia miền để viết quy tắc và hiệu suất thấp đối với các câu hỏi phức tạp hoặc không có cấu trúc rõ ràng. Trong khoảng thời gian gần đây thì các hệ thống hỏi đáp đều tiếp cận theo hướng chia hệ thống thành 2 mô-đun là (i) trích xuất thông tin và (ii) trả lời câu hỏi. Phần dưới đây em sẽ phân tích các hướng tiếp cận cho từng mô-đun này.

Trích xuất thông tin

1. **Các phương pháp không sử dụng mạng nơ-ron:** Các phương pháp này đánh giá độ liên quan thông qua sự trùng lặp từ vựng, tần suất và đặc điểm thống kê

của các từ. Hai phương pháp nổi bật là BM25 [1] và TF-IDF [2]. Nhược điểm lớn nhất của các phương pháp này là chỉ dựa trên việc so khớp từ vựng một cách thông thường mà bỏ qua thông tin về ngữ nghĩa .

2. **Các phương pháp sử dụng mạng nơ-ron:** nhằm khắc phục những hạn chế của các phương pháp ở trên về việc hiểu ngữ nghĩa. Thay vì dựa vào các đặc trưng cố định, các phương pháp này đề xuất các kiến trúc mạng nơ-ron hiệu quả và các phương pháp huấn luyện để trích xuất các mức độ trừu tượng khác nhau của ngữ nghĩa từ dữ liệu. Trước khi có BERT [3], các hệ thống chủ yếu dựa vào các mô hình nhúng từ đã được tiền huấn luyện như Word2Vec [4] và GloVe [5]. Các nhà nghiên cứu như Palangi (2016) [6] đã sử dụng mạng LSTM để nhúng câu ngữ nghĩa thông qua việc tính toán sự tương đồng giữa các câu. Shen [7] đã đề xuất một mô hình ngữ cảnh ngữ nghĩa dựa trên CNN, nén các từ thành các vector có kích thước thấp. Các mô hình theo kiến trúc Transformer [8] , đặc biệt là BERT, đã đạt được thành công đáng kể trong các nhiệm vụ xử lý ngôn ngữ tự nhiên khác nhau đặc biệt là trong lĩnh vực truy xuất thông tin nhờ vào các phương pháp cross-encoder và bi-encoder.

3. **Transformer cross-encoder:** sử dụng mã hóa chéo (cross-encoder) để mã hóa cả câu hỏi và tài liệu cùng lúc. Điều này cho phép mô hình hiểu được ngữ nghĩa của cả câu hỏi và tài liệu cũng như mối liên hệ giữa chúng một cách đồng thời. Birch [9] là một phương pháp kết hợp giữa truy xuất dựa trên từ vựng (lexical-based retrieval) và cross-encoder để trích xuất thông tin hiệu quả. Nhược điểm lớn nhất của cross-encoder là yêu cầu tài nguyên tính toán lớn và thời gian suy luận lâu.

4. **Transformer bi-encoder:** hiệu quả hơn về tài nguyên tính toán so với cross-encoder. Bi-encoder mã hóa câu hỏi và tài liệu một cách độc lập để tạo ra các vector nhúng, sau đó đánh giá độ tương đồng giữa các vector này thông qua các phép đo tương đồng. SBERT [10] là một ví dụ của bi-encoder, sử dụng SiameseBERT để tạo ra các vector nhúng câu có ý nghĩa, đạt được kết quả đáng chú ý trong các nhiệm vụ trả lời câu hỏi và truy xuất thông tin. Trong nghiên cứu của Gao [11] đã chỉ ra cơ chế chú ý nội bộ của các mô hình Transformer Encoder như BERT không tốt cho việc học biểu diễn mã hóa dày đặc, vậy nên họ đã đề xuất một kiến trúc mới là Condenser. Thời gian sau đó họ tiếp tục đề xuất kiến trúc mới là coCondenser [12] dựa trên kiến trúc Condenser và bổ sung thêm phần học biểu diễn tương phản không giám sát cấp độ văn bản trong quá trình tiền huấn luyện để warmup không gian biểu diễn.

Trả lời câu hỏi

Trước đây thì bài toán về trả lời câu hỏi thường sẽ đưa về bài toán đọc hiểu (Reading Comprehension) trong đó câu trả lời thường được rút ra trực tiếp từ đoạn văn bản cho trước. Đầu vào của bài toán Reading Comprehension là cho trước câu hỏi và một đoạn văn chứa câu trả lời của câu hỏi đó, khi đó đầu ra sẽ cần phải đưa ra vị trí đầu và cuối của câu trả lời nằm trong đoạn văn đó. Các mô hình giải quyết bài toán này bao gồm:

- Các phương pháp mô hình dựa trên LSTM với cơ chế chú ý như là: Attentive Reader [13] sử dụng cơ chế chú ý để tập trung vào các phần quan trọng của đoạn văn bản , kiến trúc Stanford Attentive Reader [14] là phiên bản nâng cao của Attentive Reader với cải tiến về cách tính toán sự chú ý. Một loại kiến trúc khác là BiDAF [15] là mô hình song song (bi-directional) kết hợp cơ chế chú ý giữa câu hỏi và đoạn văn bản. Điểm yếu của các mô hình này là yêu cầu tính toán và bộ nhớ cao, dễ bị quá khớp (overfitting) và phức tạp trong triển khai. Mặc dù đã đạt được nhiều thành công nhưng vẫn gặp khó khăn với các phụ thuộc dài hạn và tính giải thích của mô hình.
- Từ sau khi BERT ra đời thì việc tinh chỉnh BERT cũng như các mô hình tương tự như RoBERTa, DistilBERT,.. cho bài toán Reading Comprehension ngày càng nhiều. Và thực tế đã chứng minh hiệu quả của nó vượt trội hơn so với các mô hình dựa trên LSTM với cơ chế chú ý.

Tuy nhiên, nếu tiếp cận việc trả lời câu hỏi theo bài toán Reading Comprehension thì sẽ có một số điểm yếu chung. Đầu tiên, các mô hình này yêu cầu một lượng lớn dữ liệu huấn luyện có nhãn để đạt hiệu suất cao, điều này làm tăng chi phí và thời gian huấn luyện. Thứ hai, câu trả lời được trích xuất nguyên văn từ đoạn văn bản nên thường thiếu tính tự nhiên và mạch lạc, không phải lúc nào cũng phù hợp với ngữ cảnh của câu hỏi. Hơn nữa, việc huấn luyện các mô hình này cũng đòi hỏi tài nguyên tính toán lớn, khiến cho việc triển khai trở nên khó khăn hơn.

1.3 Mục tiêu và định hướng giải pháp

Mục tiêu đồ án

Qua đồ án này, em mong muốn xây dựng hệ thống hỏi đáp cho các bạn sinh viên Đại học Bách Khoa Hà Nội. Hệ thống phải đảm bảo độ chính xác cao và thời gian trả lời câu hỏi một cách nhanh chóng, có khả năng áp dụng vào thực tiễn. Từng mô-đun trong hệ thống cần khắc phục được những hạn chế đã nêu trong phần 1.2.

Định hướng giải pháp

Những giải pháp sau đây đã được em triển khai để khắc phục các vấn đề được nêu ra trong phần 1.2:

1. Khai thác các mẫu âm khó để tăng chất lượng của việc học biểu diễn tương phản.
2. Kết hợp sử dụng cross-encoder mà vẫn tránh được sự tốn kém về tài nguyên tính toán.
3. Huấn luyên đồng thời các chức năng truy xuất để tăng chất lượng của từng chức năng truy xuất này.
4. Kết hợp kết quả của nhiều mô hình với nhau để tận dụng được thế mạnh của từng mô hình.
5. Tận dụng sức mạnh suy luận của mô hình ngôn ngữ lớn mã nguồn mở để sinh câu trả lời một cách tự nhiên và chính xác cao.

1.4 Phạm vi của đồ án

Đồ án này sẽ tập trung xây dựng một hệ thống hỏi đáp hoàn chỉnh cho các bạn sinh viên Đại học Bách Khoa Hà Nội. Vậy nên em sẽ tập trung đi cải thiện cả 2 mô-đun trong hệ thống hỏi đáp là mô-đun truy xuất và mô-đun sinh câu trả lời.

Đối với mô-đun truy xuất em sẽ tiến hành thử nghiệm các phương pháp từ dựa trên thống kê thông thường cho đến những kiến trúc mô hình học sâu hiện đại và tìm cách kết hợp các phương pháp này lại để tận dụng sức mạnh của từng loại phương pháp giúp tăng độ chính xác của mô-đun truy xuất.

Đối với mô-đun sinh câu trả lời em cũng tìm hiểu và thử nghiệm nhiều kỹ thuât prompting trên các mô hình ngôn ngữ lớn mã nguồn mở khác nhau để có thể chọn được cách prompting phù hợp nhất với miền dữ liệu thông tin trong đồ án.

1.5 Đóng góp của đồ án

Đồ án này có 8 đóng góp chính như sau:

1. Thu thập và xử lý dữ liệu trên các trang thông tin chính thống của Đại học Bách Khoa Hà Nội.
2. Áp dụng kỹ thuật phân đoạn (chunking) dữ liệu để giúp nâng cao chất lượng của mô hình bi-encoder và cross-encoder trong những trường hợp văn bản cần truy xuất có kích thước lớn.
3. Sử dụng kết hợp giữa bi-encoder và cross-encoder để tận dụng được thế mạnh về tốc độ của bi-encoder cũng như độ chính xác cao của cross-encoder.
4. Sử dụng những kiến trúc Transformer mới cho mô hình bi-encoder đó là Condenser và coCondenser.
5. Thực hiện khai phá các mẫu âm khó trong nhiều vòng để tạo ra các tập dữ liệu huấn luyện hiệu quả cho việc học biểu diễn tương phản.

6. Huấn luyện đồng thời các chức năng truy xuất khác nhau để tăng chất lượng của từng chức năng truy xuất thông qua kỹ thuật tự chắt lọc tri thức (self-knowledge distillation)
7. Ensemble [16] nhiều mô hình đơn lẻ để tận dụng thể mạnh của từng mô hình.
8. Sử dụng kỹ thuật Prompting tận dụng sức mạnh của các mô hình ngôn ngữ lớn để cải thiện khả năng hiểu ngữ nghĩa và sinh câu trả lời. Với kỹ thuật này sẽ giúp hạn chế được các chi phí cao so việc huấn luyện các mô hình hỏi đáp thông thường.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

Trong chương này em sẽ giới thiệu về các chức năng truy xuất phổ biến được sử dụng trong bài toán truy xuất thông tin (Phần 2.1). Tiếp theo em sẽ trình bày về hai thuật toán được sử dụng phổ biến của chức năng truy xuất thưa thớt (Phần 2.2). Các kiến thức về học tương phản trong truy xuất thông tin cũng sẽ được em giới thiệu trong phần 2.3. Phần tiếp theo em sẽ trình bày về hai hướng tiếp cận là bi-encoder (Phần 2.4) và cross-encoder (Phần 2.5). Em cũng sẽ giới thiệu một mô hình vô cùng hiệu hiệu quả cho tác vụ truy xuất thông tin đó là M3-Embedding (Phần 2.6). Trong phần 2.7 em sẽ giới thiệu về phương pháp Ensemble cách để kết hợp kết quả của nhiều mô hình lại với nhau. Trong phần cuối cùng em sẽ trình bày về kỹ thuật prompting giúp định hướng phần sinh câu trả lời cho các mô hình ngôn ngữ lớn (Phần 2.8)

2.1 Các loại chức năng truy xuất

Truy xuất thưa thớt (Sparse retrieval)

Một biểu diễn thưa thớt (sparse representation) là một véc-tơ nhiều chiều (high-dimensional vector) mà hầu hết các giá trị bằng 0 hoặc rất nhỏ. Điều này thường xảy ra với các phương pháp truyền thống như TF-IDF và BM25 khi chỉ xét đến sự có mặt hoặc không có mặt của các từ riêng lẻ trong một tài liệu hoặc truy vấn. Các véc-tơ kết quả là thưa thớt vì chỉ một tập hợp nhỏ các từ có sẵn sẽ xuất hiện trong mỗi tài liệu hoặc truy vấn.

Các phương pháp truy xuất thưa thớt chủ yếu dựa trên sự tương đồng giữa hai văn bản, hoặc giữa một truy vấn và các văn bản dựa trên các từ khóa xuất hiện trong cả hai, vậy nên nhược điểm lớn nhất của truy xuất thưa thớt là không xem xét về mặt ngữ nghĩa của văn bản.

Truy xuất dày đặc (Dense retrieval)

Truy xuất dày đặc là một bước tiến quan trọng trong lĩnh vực tìm kiếm thông tin. Trái ngược với truy xuất thưa thớt là truy vấn dựa trên từ khóa thì truy xuất dày đặc tận dụng sức mạnh của các mô hình ngôn ngữ sâu được huấn luyện trên các tập dữ liệu lớn để hiểu sâu sắc ngữ cảnh và ngữ nghĩa của ngôn ngữ để chuyển đổi truy vấn hay tài liệu thành một vector nhúng ngữ nghĩa liên tục. Ưu điểm của truy xuất dày đặc đó là có thể hiểu và so khớp ngữ nghĩa tiềm ẩn của văn bản, dẫn đến kết quả truy vấn chính xác hơn.

Hai hướng tiếp cận phổ biến nhất theo truy xuất dày đặc đó là bi-encoder và cross-encoder.

Truy xuất đa véc-tơ (Multi-vector retrieval)

Khác so với truy xuất dày đặc mã hóa toàn bộ nội dung của một văn bản vào một véc-tơ duy nhất thì truy xuất đa véc-tơ là kỹ thuật sử dụng nhiều véc-tơ đại diện cho các đặc điểm khác nhau của một văn bản. Điều này có nghĩa là mỗi token trong một văn bản sẽ được mã hóa riêng biệt tạo ra nhiều véc-tơ đại diện.

Một kiến trúc vô cùng phổ biến theo chức năng truy xuất đa véc-tơ đó là ColBERT [17]. Mô hình này tính toán mức độ liên quan giữa truy vấn và tài liệu bằng cách chọn các token có điểm cao nhất từ tài liệu cho mỗi token của truy vấn và tổng hợp các điểm này lại. Điều này cho phép tương tác giữa các token diễn ra sau cùng, giúp áp dụng mô hình cho việc truy xuất một cách hiệu quả hơn và tận dụng tốt các tương tác mức độ chi tiết.

Truy xuất đa véc-tơ đã khắc phục được những hạn chế về khả năng biểu đạt của truy xuất dày đặc bằng cách sử dụng các tương tác chi tiết giữa các token, tuy nhiên nhược điểm là việc lưu trữ các véc-tơ cho mỗi token của tài liệu đòi hỏi chi phí lớn.

2.2 Các thuật toán truy xuất thưa thớt

Trong phần này em sẽ trình bày về hai thuật toán được dùng nhiều nhất trong truy xuất thưa thớt đó là TF-IDF và BM25 cũng như là điểm mạnh, điểm yếu của các thuật toán này .

2.2.1 TF-IDF

TF-IDF là viết tắt của "Term Frequency-Inverse Document Frequency" và là một kỹ thuật được sử dụng đầu tiên cho các tác vụ liên quan đến truy xuất thông tin. Đây là một phương pháp thống kê được sử dụng để đánh giá tầm quan trọng của một từ trong một tài liệu hoặc trong một tập hợp văn bản.

TF-IDF xem xét hai yếu tố cho từ trong một tài liệu hoặc tập hợp văn bản:

- Term Frequency (TF): dùng để đo lường tần suất của một từ trong một tài liệu hoặc tập hợp văn bản. Nó được tính bằng số lần từ xuất hiện trong tài liệu chia cho tổng số từ trong tài liệu đó.
- Inverse Document Frequency (IDF): dùng để đo lường sự hiếm gặp của một từ trong toàn bộ tập hợp văn bản. IDF được tính bằng logarit của tổng tất cả các tài liệu trong kho văn bản (corpus) chia cho số lượng tài liệu chứa từ đó.

Công thức để tính TF-IDF như sau:

$$\text{TF-IDF}(t, d, N) = \text{TF}(t, d) \times \text{IDF}(t, N) \tag{2.1}$$

Trong đó

$$\text{TF}(t, d) = \frac{\text{Số lần từ } t \text{ xuất hiện trong tài liệu } d}{\text{Tổng số từ trong tài liệu } d} \tag{2.2}$$

$$\text{IDF}(t, N) = \ln\left(\frac{N}{n_t}\right) \tag{2.3}$$

Với, N là tổng số lượng tài liệu ở trong kho văn bản (corpus) và n_t là số lượng tài liệu chứa từ t

2.2.2 BM25

BM25 (Best Match 25) là một hàm xếp hạng được sử dụng trong truy xuất thông tin và là một biến thể của thuật toán TF-IDF. Nó thường được sử dụng trong các công cụ tìm kiếm để xếp hạng sự liên quan của tài liệu đối với một truy vấn tìm kiếm cụ thể. Giống như TF-IDF, BM25 tính điểm cho mỗi tài liệu trong kho văn bản (corpus) dựa trên tần suất của các từ truy vấn trong tài liệu nhưng nó cũng tính đến độ dài của tài liệu và độ bão hòa tần suất của các từ.

Với một câu truy vấn q và một tài liệu d thì công thức tính điểm BM25 giữa q và d được tính như sau:

$$score(q, d) = \sum_{i=1}^{|q|} IDF(q_i) \cdot \frac{f(q_i, d) \cdot (k+1)}{f(q_i, d) + k \cdot (1 - b + b \cdot \frac{|d|}{avgdl})} \tag{2.4}$$

Trong đó:

- q_i là từ thứ i của câu truy vấn q
- $|q|$ là độ dài của câu truy vấn q
- $|d|$ là độ dài của tài liệu d
- $avgdl$ là độ dài trung bình của các tài liệu trong kho văn bản
- k, b là các siêu tham số
- $f(q_i, d)$ là tần suất xuất hiện của từ q_i trong tài liệu d

Và $IDF(q_i)$ được tính bởi :

$$IDF(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right) \tag{2.5}$$

với N là tổng số tài liệu trong kho văn bản và $n(q_i)$ là tổng số tài liệu có chứa từ q_i.

Công thức của BM25 khác với TF-IDF ở một vài điểm. Đầu tiên, BM25 chuẩn hóa độ dài các tài liệu thông qua $(1 - b + b \cdot \frac{|d|}{avgdl})$ và điều này cũng sẽ làm giảm trọng số của các từ thường xuyên xuất hiện trong các tài liệu dài. Thứ hai, BM25 đã chỉnh sửa công thức tính cho IDF với mục đích để thêm khả năng đưa ra score âm khi xác suất xuất hiện của từ trên toàn bộ kho văn bản rất cao (từ rất ít đặc biệt). Cuối cùng hai siêu tham số b và k có thể được tinh chỉnh cho việc chuẩn hóa độ

dài tài liệu cũng như là cái ảnh hưởng của các từ có tần suất xuất hiện quá cao khi tính toán.

2.2.3 Ưu và nhược điểm của TF-IDF và BM25

Mặc dù các thuật toán theo truy xuất thưa thớt đã xuất hiện từ rất sớm (TF-IDF được ra đời năm 1973 và BM25 được phát triển vào những năm 1980) nhưng đến nay vẫn được sử dụng rất phổ biến, đặc biệt là BM25, nó hầu như đều được sử dụng như một baseline dùng để so sánh trong các nghiên cứu về lĩnh vực truy xuất thông tin. Ưu điểm nổi bật của các thuật toán này bao gồm :

- Vô cùng đơn giản, dễ dàng triển khai và rất hiệu quả
- Việc tính toán trong trong các thuật toán này chỉ đơn thuần dựa trên thông kê nên có tốc độ xử lý rất nhanh và sẽ rất hữu dụng trong các tình huống mà số lượng các tài liệu trong kho văn bản rất lớn.
- Đặc biệt là BM25 có sử dụng chuẩn hóa độ dài các tài liệu, điều này giúp giải quyết được vấn đề gặp phải rất nhiều trong thực tế đó là bị lệch độ dài tài liệu.

Đi kèm với những ưu điểm trên thì những thuật toán này vẫn còn những điểm hạn chế như là :

- Hai thuật toán này đều không tính đến ngữ cảnh của các từ khóa trong truy vấn tìm kiếm.
- Do dựa hoàn toàn theo phương pháp thông kê nên chúng có thể bỏ qua một số yếu tố quan trọng khác như cấu trúc văn bản, cái mà đóng vai trò quan trọng trong việc thể hiện ý nghĩa và mối quan hệ giữa các từ.

Những giới hạn này có thể được vượt qua bằng cách sử dụng các thuật toán cũng như các kỹ thuật xử lý ngôn ngữ tự nhiên hiện đại hơn. Tuy nhiên các thuật toán này vẫn là nền móng vô cùng quan trọng trong quá trình phát triển lĩnh vực truy xuất thông tin.

2.3 Học tương phản trong truy xuất thông tin

Học tương phản (Contrastive Learning) là một phương pháp học sâu mạnh mẽ nhằm cải thiện khả năng phân loại và truy xuất thông tin. Phương pháp này tập trung vào việc học các biểu diễn véc-tơ của dữ liệu sao cho các mẫu tương đồng được mã hóa gần nhau trong không gian biểu diễn, trong khi các mẫu không tương đồng được mã hóa xa nhau.

2.3.1 Nguyên lý của học tương phản

Học tương phản sử dụng các cặp mẫu dương (positive samples) và mẫu âm (negative samples) để tối ưu hóa biểu diễn của dữ liệu. Mục tiêu là để các mẫu

cùng lớp (dương) gần nhau và các mẫu khác lớp (âm) xa nhau trong không gian nhúng.

Hàm mất mát tương phản thường được sử dụng để đạt được mục tiêu này. Một phiên bản phổ biến của hàm mất mát tương phản được định nghĩa như sau:

$$\mathcal{L}_{cont}(x_i, x_j, \theta) = \mathbb{I}[y_i = y_j]\|f_o(x_i) - f_o(x_j)\|_2^2 + \mathbb{I}[y_i \neq y_j]\max(0, \epsilon - \|f_o(x_i) - f_o(x_j)\|_2^2) \tag{2.6}$$

Trong đó:

- $\mathbb{I}$ là hàm chỉ thị, với $\mathbb{I}[y_i = y_j]$ bằng 1 nếu $y_i = y_j$ và bằng 0 nếu $y_i \neq y_j$.
- y_i và y_j là nhãn của các mẫu x_i và x_j.
- $f_o(x)$ là hàm mã hóa vector.
- ϵ là siêu tham số, xác định khoảng cách tối thiểu giữa các vector của các mẫu từ các lớp khác nhau.
- $\|.\|_2$ là chuẩn Euclid (L2 norm) của vector.

2.3.2 Các kỹ thuật thường được sử dụng trong học tương phản

Hai kỹ thuật giúp cho việc học tương phản trở nên hiệu quả đó là (i) kích thước lô lớn và (ii) khác thác mẫu âm khó.

Kích thước lô lớn: Vì trong học tương phản các mẫu dương và các mẫu âm sẽ được đưa vào trong 1 lô, vậy nên nếu kích thước lô đủ lớn thì sẽ giúp bao phủ đa dạng các mẫu âm và mẫu dương và giúp cho việc học tương phản trở nên hiệu quả hơn.

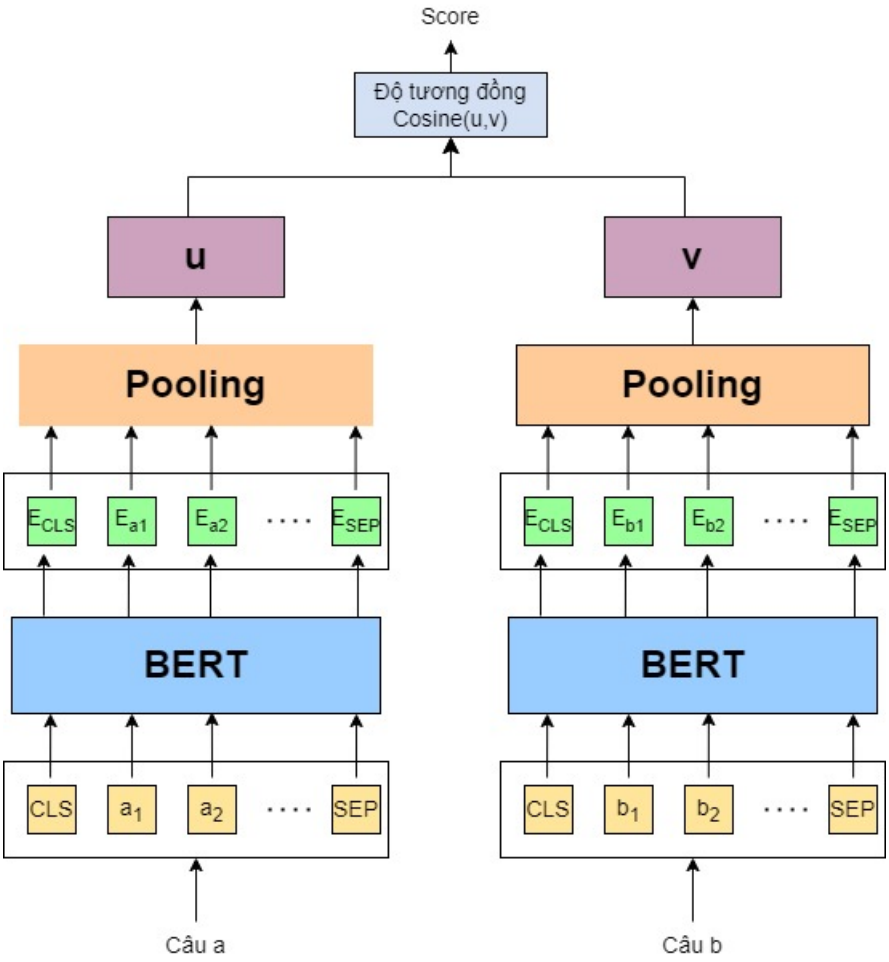
Khai thác mẫu âm khó: Các mẫu âm khó là các mẫu có nhãn khác nhưng lại có véc-tơ biểu diễn gần với mẫu dương, nếu khai thác tốt các mẫu âm khó thì sẽ giúp mô hình học phân biệt tốt hơn và chống lại sự nhầm lẫn ngữ nghĩa.

2.4 Hướng tiếp cận Bi-encoder

Kể từ khi ra đời thì Transformer đã thể hiện khả năng học tập ngữ cảnh xa và xử lý các chuỗi văn bản dài hiệu quả vượt trội so với các mô hình trước đây. Một mô hình dựa trên kiến trúc Transformer Encoder đó là BERT, đây là một công cụ vô cùng mạnh mẽ trong việc hiểu ngữ nghĩa, ngữ cảnh của câu và mã hóa câu thành một véc-tơ nhúng. Và ngày càng nhiều các mô hình Transformer Encoder đã được tiền huấn luyện và chúng trở thành các bộ mã hóa văn bản vô cùng hiệu quả. Trong các tác vụ về truy xuất thông tin thì bi-encoder là một trong những kiến trúc Transformer Encoder được sử dụng phổ biến nhất.

Kiến trúc chung của bi-encoder được thể hiện trong hình 2.1. Đầu vào của kiến trúc bi-encoder bao gồm hai câu , giả sử là a và b. Đầu tiên, hai câu a và b được

chuyển thành các token, sau đó các token sẽ được đưa qua một mô hình Transformer Encoder như BERT để mã hóa và thu được hai danh sách các véc-tơ nhúng tương ứng. Hai danh sách các véc-tơ nhúng này được đưa qua lớp pooling để thu được hai véc-tơ nhúng biểu diễn cho hai câu a và b. Và độ tương đồng của hai câu đầu vào a và b được tính bằng độ đo cosine giữa hai véc-tơ nhúng biểu diễn cho hai câu đó.



Hình 2.1: Kiến trúc bi-encoder

2.4.1 Tiền huấn luyện Masked Language Model

Các mô hình Transformer Encoder như BERT, RoBERTa,... đều được tiền huấn luyện với tác vụ Masked Language Model, đặc biệt trong một miền dữ liệu cụ thể thì việc tiền huấn luyện Masked Language Model sẽ giúp cho mô hình có thể thích nghi được với miền dữ liệu đó.

Trong Masked Language Model, một tỉ lệ phần trăm nhất định các từ trong câu bị ẩn đi, các từ này được che bởi một token đặc biệt đó là [MASK] token và sau đó mô hình phải có nhiệm vụ là dự đoán ra các từ bị ẩn đi này dựa trên các từ còn lại trong câu.

Hình 2.2 biễn diễn cách Masked Language Model hoạt động. 15% các từ trong

câu sẽ được chọn để bị che đi. Mô hình sẽ cần phải học cách chọn một từ chính xác để thay thế cho các từ bị che trong câu đầu vào. Sau khi cho qua các khối encoder thì các véc-tơ nhúng đại diện cho các [MASK] token sẽ được cho đi qua mạng nơ-ron truyền thẳng (feed forward neural network) và lớp softmax để xác định từ nào trong bộ từ vựng (vocabulary) là ứng viên tốt cho vị trí bị che.

Cho đầu vào với tập hợp $T = t_1, t_2, ..., t_N$ gồm N token. Mô hình sẽ che đi token t_i tại vị trí i và i sẽ được lưu trong tập M, tập M này chứa chỉ số của các token bị che trong T. Hàm mất mát được sử dụng là cross-entropy và được tính như sau:

$$\hat{y}_i = W h_i^L \tag{2.7}$$

$$\mathcal{L}_{mlm} = \sum_{i \in M} \text{CrossEntropy}(\hat{y}_i, t_i) \tag{2.8}$$

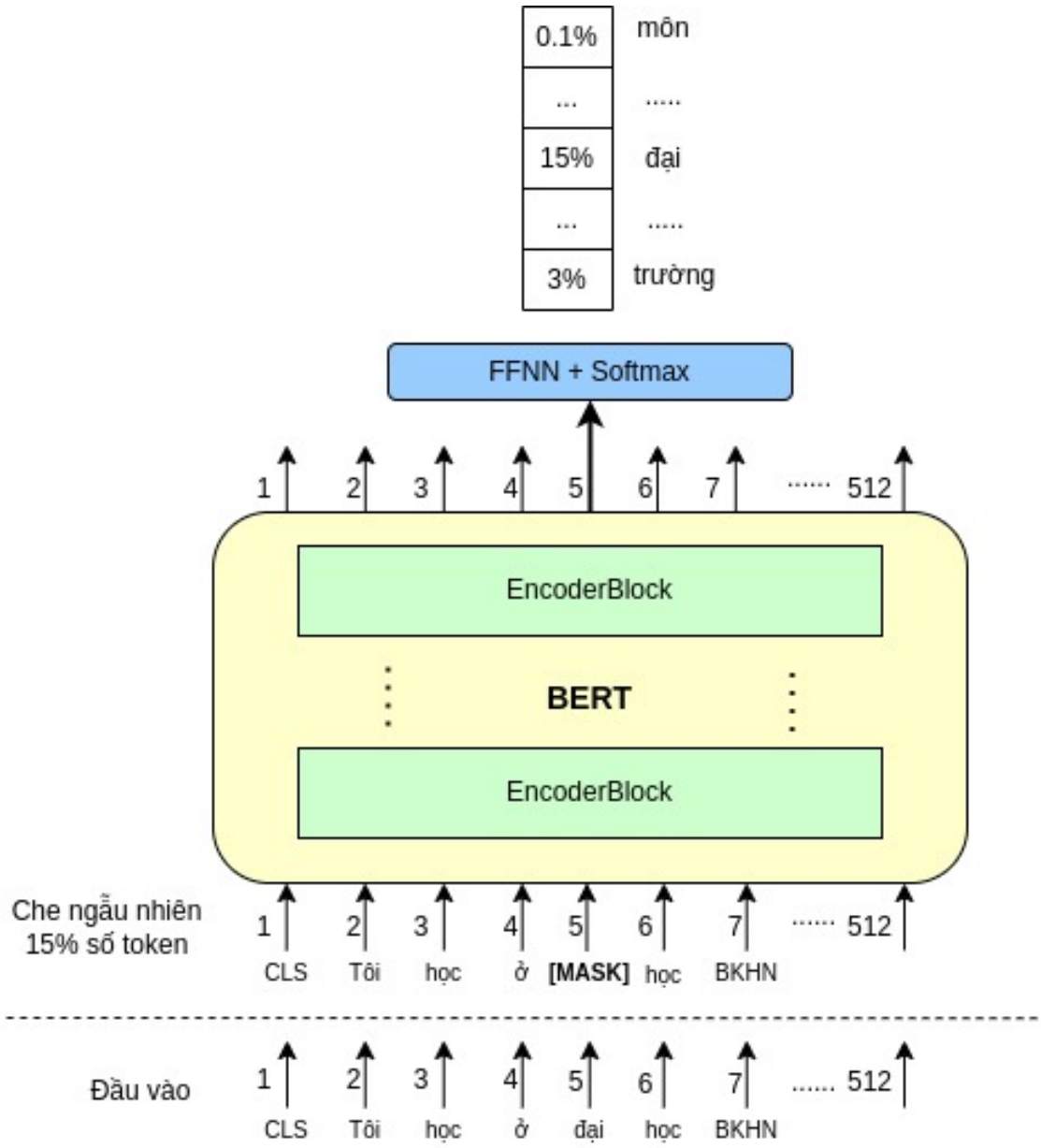
trong đó:

- h_i^L là véc-tơ nhúng đại diện cho token bị che tại vị trí i và lớp thứ L.
- $\hat{y}_i$ là kết quả đầu ra sau khi đi qua mạng rơ-ron truyền thẳng và lớp softmax.

2.4.2 Condenser và phương pháp cải tiến

Trong các tác vụ về truy xuất thông tin, các mô hình Transformer Encoder đã được tiền huấn luyện sẽ được thực hiện tinh chỉnh để mã hóa câu truy vấn và các đoạn văn bản thành các véc-tơ nhúng biểu diễn. Tuy nhiên để tinh chỉnh các mô hình Transformer Encoder này một cách hiệu quả thì yêu cầu số lượng dữ liệu để đào tạo phải lớn và thứ hai đó là yêu cầu các kĩ thuật phức tạp. Để tìm hiểu vấn đề này thì cần phải xem xét kiến trúc của các mô hình Transformer Encoder này. BERT là một mô hình Transformer Encoder và trong BERT tất cả các token, bao gồm token đặc biệt CLS - là token được sử dụng để tổng hợp thông tin toàn bộ câu, đều được nhận thông tin từ các token khác trong chuỗi thông qua cơ chế chú ý (attention). Trong các lớp giữa của BERT, token CLS có cách thức chú ý tương tự như các token khác và các token khác cũng không chú ý nhiều đến token CLS, điều này có nghĩa là token CLS không thực sự được sử dụng để tổng hợp thông tin từ các token khác ở các lớp giữa này. Khi đến các lớp cuối cùng của BERT, token CLS bắt đầu có sự chú ý rộng hơn, tức là nó bắt đầu chú ý vào toàn bộ câu. Điều này là cần thiết để thực hiện nhiệm vụ Next Sentence Prediction (NSP), một trong những nhiệm vụ mà BERT được tiền huấn luyện.

Trong bài báo Condenser, các tác giả đã cho thấy rằng một bộ mã hóa bi-encoder hiệu quả nên tổng hợp thông tin ở nhiều mức độ khác nhau từ toàn bộ câu qua tất cả các lớp, thay vì chỉ ở các lớp cuối cùng như cách mà token CLS trong kiến trúc BERT hiện tại đang làm. Chính vì vậy các tác giả họ đã đề xuất ra Condenser, một



Hình 2.2: Ví dụ về Masked Language Model

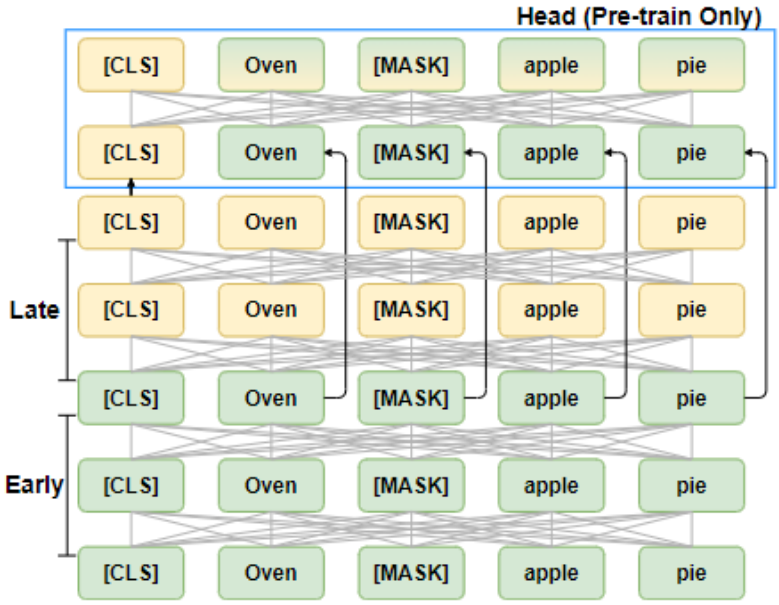
kiến trúc dựa trên Transformer Encoder và nhằm tiền huấn luyện cho tác vụ truy xuất thông tin.

Kiến trúc mô hình Condenser

Condenser được xây dựng dựa trên kiến trúc của BERT và thêm các lớp Condenser head. Hình 2.3 mô tả Kiến trúc của Condenser và em trích ra từ bài báo gốc của Condenser [11]

Có 3 lớp chính trong kiến trúc của Condenser đó là (i)L^e là các lớp encoder early của BERT,(ii) L^l là các lớp encoder cuối late của BERT và (iii) L^h là các lớp Condenser head. Đầu tiên mô hình sẽ mã hóa đầu vào bởi các khối Encoder của BERT:

$$[h_{cls}^{early}, h^{early}] = \text{Encoder}_{early}([h_{cls}^{0}, h^{0}]) \tag{2.9}$$



Hình 2.3: Kiến trúc Condenser

$$[h_{cls}^{late}, h^{late}] = \text{Encoder}_{late}([h_{cls}^{early}, h^{early}]) \tag{2.10}$$

Điểm đặc biệt nhất của Condenser là thêm một nối tắt từ đầu ra của lớp encoder early đến lớp Condenser head:

$$[h_{cls}^{cd}, h^{cd}] = \text{Condenser}_{head}([h_{cls}^{late}, h^{early}]) \tag{2.11}$$

Condenser được tiền huấn luyện với hàm mất mát Masked Language Model với đầu ra của lớp Condenser head:

$$\mathcal{L}_{mlm} = \sum_{i \in \text{masked}} \text{CrossEntropy}(W h_i^{cd}, x_i) \tag{2.12}$$

Từ kiến trúc của Condenser ở trên thì việc thêm một nối tắt từ các lớp Encoder early đã giúp giảm bớt đi phần mã hóa các thông tin chi tiết của văn bản đầu vào từ đó token CLS có thể tập trung vào việc học ý nghĩa tổng quát của văn bản.

Tiền huấn luyện Condenser

Để giúp giảm chi phí cho việc tiền huấn luyện Condenser thì tại các lớp early và late của Condenser được khởi tạo bằng với trọng số của BERT đã được tiền huấn luyện và trọng số tại các lớp Condenser head sẽ được khởi tạo ngẫu nhiên. Với việc khởi tạo ngẫu nhiên tại các lớp head này nếu thực hiện lan truyền ngược từ các lớp head thì sẽ làm ảnh hưởng đến trọng số của các lớp early và late, vì vậy Condenser đã giải quyết vấn đề này bằng việc thêm một ràng buộc là tính toán hàm mất mát

Masked Language Model với cả đầu ra của lớp late:

$$\mathcal{L}^c_{mlm} = \sum_{i \in \text{masked}} \text{CrossEntropy}(W h_i^{late}, x_i) \tag{2.13}$$

Hàm mất mát cuối cùng sẽ được tính như sau:

$$\mathcal{L} = \mathcal{L}_{mlm} + \mathcal{L}^c_{mlm} \tag{2.14}$$

Tinh chỉnh Condenser

Trong quá trình tinh chỉnh thì các lớp Condenser head sẽ bị loại bỏ và huấn luyện token CLS late h^{late}_{cls} cũng như thực hiện quá trình lan truyền ngược về các lớp của BERT. Chính bởi vậy Condenser chính là kiến trúc Transformer Encoder trong quá trình tinh chỉnh.

Phương pháp cải tiến cho Condenser

Trong bài báo về coCondenser [12] các tác giả đã chỉ ra hai vấn đề chính gặp phải khi tinh chỉnh BERT và Condenser trong truy xuất thông tin đó là:

- Dễ dàng bị ảnh hưởng bởi dữ liệu nhiễu.
- Cần kích thước lô dữ lớn để học không gian nhúng một cách hiệu quả.

Chính bởi vậy họ đã đề xuất ra một hướng phát triển tiếp theo của Condenser là coCondenser, với coCondenser sẽ không cần phải có các quy trình kỹ thuật dữ liệu phức tạp như tăng cường dữ liệu (augmentation), tổng hợp (synthesis), hay lọc (filtering), cũng như không cần huấn luyện với kích thước lô lớn. Họ cũng chứng minh được sự hiệu quả của coCondenser khi so sánh hiệu suất của nó tương đương với RocketQA [18] - một hệ thống hiện đại và phức tạp, chỉ với việc tinh chỉnh đơn giản với kích thước lô nhỏ.

Kiến trúc của coCondenser được thiết kế giống với Condenser và coCondenser có bổ sung thêm một hàm mất mát tương phản ở cấp độ kho văn bản. Hàm mất mát này được sử dụng trực tiếp để huấn luyện không gian nhúng của các đoạn văn bản. Cách tiếp cận này sẽ giúp warmup không gian nhúng và tạo ra một không gian nhúng một cách hiệu quả. Giả sử, có danh sách ngẫu nhiên gồm n tài liệu $[d_1, d_2, ..., d_n]$, trích xuất từ mỗi tài liệu ngẫu nhiên một cặp đoạn văn $[s_{11}, s_{12}, ..., s_{n1}, s_{n2}]$. Các đoạn văn này sau đó tạo thành một lô huấn luyện của coCondenser. Giả sử h_{ij} là véc-tơ nhúng biểu diễn cho token CLS ở lớp late của đoạn văn bản s_{ij}, khi đó hàm mất mát tương phản được xác định như sau:

$$\mathcal{L}^{\text{co}}_{ij} = -\log \frac{\exp(\langle h_{i1}, h_{i2} \rangle)}{\sum_{k=1}^{n} \sum_{l=1}^{2} \mathbb{I}_{ij \neq kl} \exp(\langle h_{ij}, h_{kl} \rangle)} \tag{2.15}$$

Công thức của hàm mất mát trên dựa trên giả định của các tác giả trong bài báo này đó là những đoạn văn mà được lấy ra từ cùng một tài liệu thì nên có véc-tơ biểu diễn gần nhau và những đoạn văn từ những tài liệu khác nhau thì nên có véc-tơ biểu diễn xa nhau. Giả sử hàm mất mát Masked Language Model của đoạn văn s_{ij} là $\mathcal{L}_{ij}^{mlm}$, khi đó hàm mất mát của cả lô được tính như sau:

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^{n} \sum_{j=1}^{2} \left[\mathcal{L}_{ij}^{\text{mlm}} + \mathcal{L}_{ij}^{\text{co}} \right] \tag{2.16}$$

Sử dụng bộ nhớ hiệu quả trong quá trình tiền huấn luyện coCondenser

Để huấn luyện việc học tương phản không giám sát một cách hiệu quả thì thường yêu cầu kích thước lô phải lớn và việc huấn luyện với kích thước lớn thường sẽ không thể thực hiện được với phần cứng phổ thông. Trong coCondenser các tác giả đã sử dụng một kỹ thuật là gradient caching với ý tưởng là tách rời việc lan truyền ngược giữa hàm mất mát và bộ mã hóa và sử dụng bộ đệm để lưu trữ tính toán gradient tại các bước trung gian .

Tinh chỉnh coCondenser

Trong quá trình tinh chỉnh coCondenser thì các lớp Condenser head sẽ bị loại bỏ và trọng số của mô hình backbone được dùng để khởi tạo bộ mã hóa truy vấn (query encoder) f_q và bộ mã hóa đoạn văn (passage encoder) f_p. Khi đó độ tương đồng giữa một cặp (truy vấn q và đoạn văn p) sẽ được tính bởi:

$$s(q, p) = \langle f_q(q), f_p(p) \rangle \tag{2.17}$$

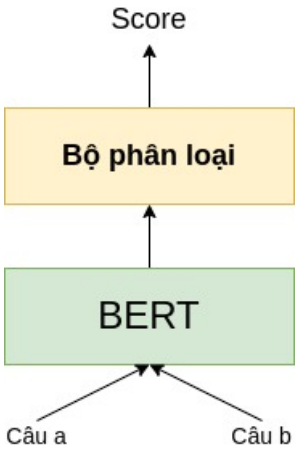
Các bộ mã hóa truy vấn và mã hóa đoạn văn này sẽ được tinh chỉnh với hàm mất mát tương phản có giám sát. Với truy vấn q, tính negative log likelihood của một tài liệu dương d^+ với tập tài liệu âm $\{d_1^-, d_2^-, ..., d_l^-, ...\}$

$$\mathcal{L} = -\log \frac{\exp(s(q, d^+))}{\exp(s(q, d^+)) + \sum_l \exp(s(q, d_l^-))} \tag{2.18}$$

2.5 Hướng tiếp cận Cross-encoder

Một kiến trúc cũng được sử dụng nhiều trong các bài toán về truy xuất thông tin đó là cross-encoder. Kiến trúc của cross-encoder được thể hiện trong hình 2.4. Đầu vào của mô hình là một cặp (Câu a, Câu b), sau đó mỗi câu này được tách thành các token và tiếp tục nối hai danh sách các token này bởi token [SEP]. Cho toàn bộ đồng thời các token sau khi được nối này qua BERT để thu được các véc-tơ nhúng tương ứng với từng token. Véc-tơ biểu diễn cho cặp câu đầu vào được

tính bằng cách thực hiện gộp các véc-tơ nhúng này theo mean-pooling. Cuối cùng véc-tơ biểu diễn cho cặp câu này sẽ được cho qua một bộ phân loại để trả về điểm (Score) biểu thị sự tương đồng giữa hai câu. Chính bởi việc cho đồng thời cặp câu vào BERT và dựa vào cơ chế chú ý của BERT đã giúp cho việc hiểu ngữ cảnh cũng như là cái mỗi liên hệ giữa cặp câu đó, và do vậy nên độ chính xác của kiến trúc cross-encoder thường cao hơn so với kiến trúc bi-encoder trong các bài toán truy xuất thông tin. Bên cạnh độ chính xác cao thì cross-encoder cũng có nhược



Hình 2.4: Kiến trúc cross-encoder

điểm về vấn đề tốc độ khi số lượng cặp các văn bản truy xuất tăng lên cao. Giả sử có một tập hợp gồm n = 100000 câu, khi đó để tìm được cặp câu có độ tương đồng cao nhất ta cần phải thực hiện suy luận trên $\frac{n!}{2!(n-2)!}$ = 4999950000 cặp câu đối với cross-encoder, trong khi đó đối với bi-encoder ta chỉ cần thực hiện mã hóa cho 100000 câu đó. Chính bởi vậy nên trong các hệ thống truy xuất thông tin người ta thường kết hợp cả bi-encoder và cross-encoder để khắc phục nhược điểm của bi-encoder.

2.6 M3-Embedding

M3-Embedding là một mô hình nhúng tiên tiến với nhiều tính năng vượt trội và đặc biệt là trong truy xuất thông tin. Được thiết kế với khả năng đa ngôn ngữ, M3-Embedding hỗ trợ hơn 100 ngôn ngữ khác nhau. Mô hình này nổi bật với khả năng thực hiện đồng thời ba chức năng truy xuất quan trọng đã giúp tạo ra một nền tảng mô hình thống nhất, linh hoạt trong việc xử lý các yêu cầu truy xuất thông tin khác nhau. Đặc điểm nổi bật nữa đó là khả năng xử lý các đầu vào từ câu ngắn đến các tài liệu dài lên đến 8192 tokens, M3-Embedding đáp ứng tốt các yêu cầu về độ chi tiết khác nhau trong dữ liệu.

2.6.1 Truy xuất kết hợp (Hybrid retrieval)

M3-Embedding thống nhất cả ba chức năng truy xuất phổ biến của mô hình nhúng bao gồm truy xuất dày đặc (dense retrieval), truy xuất thưa thớt (lexical (sparse) retrieval) và truy xuất đa véc-tơ (multi-vector retrieval). Công thức của từng loại được tính như sau

Dense Retrieval

Câu truy vấn q được chuyển thành các trạng thái ẩn (hidden states) dựa trên một bộ mã hóa văn bản và trạng thái ẩn sau khi được chuẩn hóa của token đặc biệt "[CLS]" sẽ dùng làm véc-tơ biểu diễn cho câu truy vấn: $e_q = \text{norm}(H_q[0])$. Tương tự, cũng có thể lấy được véc-tơ biểu diễn của đoạn văn p là $e_p = \text{norm}(H_p[0])$. Do đó, điểm tương quan giữa câu truy vấn và đoạn văn được đo bằng tích vô hướng giữa hai véc-tơ embedding e_q và e_p là :

$$S_{\text{dense}} \leftarrow \langle e_p, e_q \rangle \tag{2.19}$$

Lexical Retrieval

Các phần nhúng đầu ra cũng có thể sử dụng để ước tính sự quan trọng của từng từ trong truy xuất từ vựng. Đối với mỗi từ t trong câu truy vấn thì trọng số của từ được tính bằng: $w_{q_t} \leftarrow \text{Relu}(W_{\text{lex}}^T H_q[i])$. Nếu một từ t xuất hiện nhiều lần trong câu truy vấn thì sẽ lấy trọng số lớn nhất. Điểm tương quan giữa câu truy vấn và đoạn văn được tính bằng mức độ quan trọng chung của các từ cùng tồn tại trong câu truy vấn và đoạn văn đó và được tính bằng :

$$S_{\text{lex}} \leftarrow \sum_{t \in q \cap p} (w_{q_t} * w_{p_t}) \tag{2.20}$$

Multi-Vec Retrieval

Truy xuất đa véc-tơ là phần mở rộng của truy xuất dày đặc, phương pháp này sử dụng toàn bộ phần nhúng đầu ra làm biểu diễn cho câu truy vấn và đoạn văn: $E_q = \text{norm}(W_{\text{mul}}^T H_q)$, $E_p = \text{norm}(W_{\text{mul}}^T H_p)$. Điểm tương quan giữa câu truy vấn và đoạn văn được tính theo phương pháp ColBert [17] đó là sử dụng các tương tác muộn (late-interaction) :

$$S_{\text{mul}} \leftarrow \frac{1}{N} \sum_{i=1}^{N} \max_{j=1}^{M} E_q[i] \cdot E_p^T[j], \tag{2.21}$$

với N, M lần lượt là độ dài của câu truy vấn và đoạn văn.

Truy xuất kết hợp có thể áp dụng vào trong quá trình truy xuất như sau: đầu tiên, các kết quả ứng viên có thể được truy xuất riêng lẻ bằng từng phương pháp, sau đó

kết quả truy xuất cuối cùng sẽ được xếp hạng lại dựa trên điểm tương quan quan tích hợp:

$$S_{\text{rank}} \leftarrow S_{\text{dense}} + S_{\text{lex}} + S_{\text{mul}} \tag{2.22}$$

2.6.2 Self-Knowledge Distillation

Mô hình nhúng được huấn luyện để phân biệt giữa các mẫu dương và các mẫu âm vì vậy nên quá trình huấn luyện nhằm mục đích giảm thiểu hàm mất mát InfoNCE với công thức tổng quát là :

$$\mathcal{L} = -\log \frac{\exp(S(q, p^*)/\tau)}{\sum_{p \in \{p^*, P'\}} \exp(S(q, p)/\tau)} \tag{2.23}$$

với p^* và P' đại diện cho các mẫu dương và âm cho truy vấn q và $S(\cdot)$ là bất kì hàm nào trong $\{S_{\text{dense}}(\cdot), S_{\text{lex}}(\cdot), S_{\text{mul}}(\cdot)\}$.

Một vấn đề gặp phải đó là mục tiêu huấn luyện của các phương pháp truy xuất khác nhau có thể gây xung đột lẫn nhau dẫn đến việc huấn luyện đa mục tiêu nguyên gốc có thể làm giảm chất lượng của việc nhúng. Để giải quyết được vấn đề này, các tác giả của bài báo M3-Embedding đã đề xuất hợp nhất quá trình huấn luyện dựa trên **self-knowledge distillation**. Phương pháp này dựa trên nguyên tắc của ensemble learning, các dự đoán từ các phương pháp truy xuất khác nhau có thể được tích hợp lại để tạo ra một điểm tương quan chính xác hơn nhờ vào tính chất không đồng nhất của chúng. Ở dạng đơn giản nhất, sự tích hợp này chỉ cần là tổng hợp của các điểm dự đoán khác nhau:

$$S_{\text{inter}} \leftarrow S_{\text{dense}} + S_{\text{lex}} + S_{\text{mul}} \tag{2.24}$$

Lúc này điểm tích hợp S_{inter} như một teacher và hàm mất mát của mỗi phương pháp truy xuất thưa thớt sẽ được sửa đổi thành:

$$\mathcal{L}'_* \leftarrow -p(S_{\text{inter}}) * \log p(S_*) \tag{2.25}$$

Trong đó $p(.)$ là hàm kích hoạt softmax, S_* là bất kì trong S_{dense}, S_{lex}, S_{mul} . Sau đó hàm mất mát tích hợp và chuẩn hóa như sau:

$$\mathcal{L}' \leftarrow \frac{\mathcal{L}'_{\text{dense}} + \mathcal{L}'_{\text{lex}} + \mathcal{L}'_{\text{mul}}}{3} \tag{2.26}$$

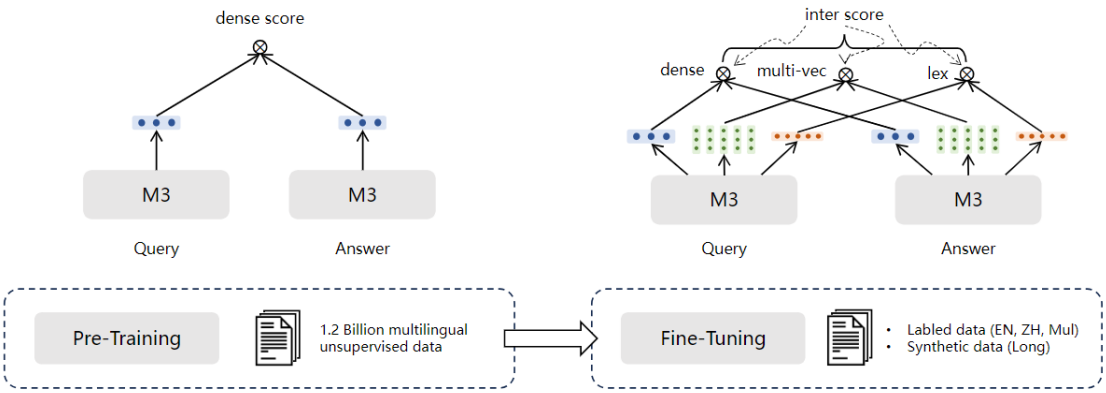
Và hàm mất mát cuối cùng cho self-knowledge distillation với sự kết hợp tuyến tính của $\mathcal{L}$ và $\mathcal{L}'$:

$$\mathcal{L}_{\text{final}} \leftarrow \mathcal{L} + \mathcal{L}' \tag{2.27}$$

2.6.3 Huấn luyện mô hình M3-Embedding

Quá trình huấn luyện tổng thể của mô hình M3-Embedding bao gồm 2 giai đoạn được trình bày trong hình 2.5 được em trích dẫn từ bài báo gốc của mô hình M3-Embedding [19].

Hai giai đoạn huấn luyện của M3-Embedding bao gồm:



Hình 2.5: Quy trình huấn luyện của M3-Embedding với tự chắt lọc tri thức

1. Giai đoạn đầu tiên, bộ mã hóa văn bản được huấn luyện với một lượng dữ liệu không giám sát lớn, trong đó chỉ có truy xuất dày đặc được huấn luyện dưới dạng cơ bản của học tương phản. Trong đó bộ mã hóa văn bản sử dụng mô hình hỗ trợ đa ngôn ngữ là XLM-RoBERTa [20] được tiền huấn luyện thông qua phương pháp RetroMAE [21].

2. Giai đoạn thứ hai áp dụng quá trình tự chắt lọc tri thức, trong đó mô hình nhúng được tinh chỉnh để thiết lập ba chức năng truy xuất là truy xuất dày đặc, truy xuất thưa thớt và truy xuất đa véc-tơ.

2.7 Phương pháp Ensemble

Ensemble là một kỹ thuật trong học máy sử dụng nhiều mô hình khác nhau để cải thiện độ chính xác và hiệu suất của hệ thống. Thay vì dựa vào một mô hình duy nhất, phương pháp này tổng hợp kết quả từ nhiều mô hình để tạo ra một dự đoán cuối cùng chính xác hơn. Bằng cách tận dụng sức mạnh của các mô hình đơn lẻ, ensemble có thể giảm thiểu sai số và tăng cường khả năng tổng quát hóa.

Để cho phương pháp Ensemble trở nên hiệu quả thì yếu tố quan trọng nhất đó là làm sao để kết hợp dự đoán của các mô hình khác nhau một cách hiệu quả. Có 3 kỹ thuật chính được sử dụng phổ biết nhất bao gồm:

1. **Majority Voting (Bình chọn đa số):** Phương pháp này thường được sử dụng trong các bài toán về phân loại, trong đó nhiều mô hình sẽ được sử dụng để dự đoán cho từng điểm dữ liệu. Những dự đoán nhận được từ phần lớn các mô

hình sẽ được sử dụng làm dự đoán cuối cùng.

2. **Averaging (Lấy giá trị trung bình):** Phương pháp này thường được sử dụng trong các bài toán hồi quy. Kết quả cuối cùng là giá trị trung bình của tất cả các dự đoán từ các mô hình.

3. **Weighted Averaging (Lấy giá trị trung bình có trọng số):** Trong quá trình ensemble không phải mô hình cũng có độ tin cậy giống nhau do đó chúng cần được gán trọng số khác nhau trong quá trình tổng hợp. Trọng số phản ánh mức độ quan trọng của mỗi mô hình trong dự đoán cuối cùng.

2.8 Kỹ thuật prompting

Kỹ thuật prompting liên quan đến việc cung cấp các lời gợi ý (prompts) để hướng dẫn mô hình AI tạo ra phản hồi hoặc thực hiện một nhiệm vụ cụ thể.

Kỹ thuật prompting hoạt động bằng cách cung cấp cho mô hình ngôn ngữ một đoạn văn bản ban đầu, được gọi là prompt. Prompt này có thể là một câu hỏi, một đoạn mô tả, hoặc bất kỳ đoạn văn nào khác nhằm hướng dẫn mô hình để tạo ra nội dung tiếp theo. Dựa trên prompt được cung cấp, mô hình AI sẽ sử dụng các kiến thức đã được huấn luyện để tạo ra phản hồi.

Các kỹ thuật prompting phổ biến

1. **Zero-Shot Prompting**
   Đây là một phương pháp nhằm cải thiện zero-shot learning trong các mô hình ngôn ngữ lớn (LLMs). Học zero-shot giúp mô hình có khả năng nắm bắt và thực hiện những nhiệm vụ mà nó chưa từng được huấn luyện một cách cụ thể. Phương pháp này bao gồm việc tinh chỉnh các mô hình ngôn ngữ đã được huấn luyện trước trên một loạt các nhiệm vụ NLP được mô tả trong prompt bằng ngôn ngữ tự nhiên. Cách tiếp cận này đã được chứng minh là cải thiện đáng kể khả năng của mô hình trong việc thực hiện các nhiệm vụ chưa được nhìn thấy (unseen tasks). Hiệu quả của việc tinh chỉnh prompt được thể hiện qua sự cải thiện hiệu suất đáng kể so với các mô hình ngôn ngữ, lợi ích càng trở nên rõ rệt khi số lượng nhiệm vụ tăng lên và khi được áp dụng với LLMs. Trong nghiên cứu [22] Wei et al., đã kết luận rằng việc tinh chỉnh prompt là một kỹ thuật triển vọng để cải thiện khả năng tổng quát hóa và hiệu suất của LLMs trong các tình huống zero-shot learning.

2. **Few-Shot Prompting**
   Few-shot learning được giới thiệu trong [23] như một cách để mô hình hóa các mô hình ngôn ngữ có thể thích nghi và thực hiện các nhiệm vụ dựa trên một số ít ví dụ cụ thể được cung cấp trong quá trình suy luận, mà không cần cập nhật trong số mô hình hay tinh chỉnh cụ thể cho nhiệm vụ đó. Điều này

cho phép mô hình hoạt động gần giống với cách con người học đó là có thể học và thực hiện tác vụ mới với chỉ vài hướng dẫn cụ thể. Kết quả từ bài báo cho thấy mô hình GPT-3 có khả năng thực hiện khá tốt trên nhiều tập dữ liệu với few-shot, thỉnh thoảng ngang bằng hoặc thậm chí vượt qua hiệu suất của các mô hình được tinh chỉnh cụ thể . Điều này cho thấy tiềm năng lớn của việc sử dụng few-shot learning để cải thiện khả năng tổng quát hóa và thích ứng nhanh của các mô hình ngôn ngữ lớn với các nhiệm vụ mới mà chúng không được tinh chỉnh trực tiếp.

Với Few-Shot Prompting có thể một ví dụ duy nhất (học 1-shot) đối với một số tác vụ dễ, đối với một số tác vụ khó khăn hơn thì có thể thử nghiệm bằng cách tăng số lượng mẫu (3-shot, 5-shot, 10-shot,...).

3. **Chain-of-Thought Prompting**

   Phương pháp "*Chain of Thought*" (COT) được giới thiệu như một cách để cải thiện hiệu suất của LLMs trong việc giải quyết các bài toán đòi hỏi suy luận phức tạp. Phương pháp này dựa trên việc kích thích mô hình tạo ra một chuỗi các bước suy nghĩ trung gian tự nhiên, giúp dẫn đến kết quả cuối cùng. Điều này cho phép mô hình phân rã các vấn đề thành các bước nhỏ hơn, làm cho quá trình suy luận trở nên minh bạch và dễ hiểu hơn.

Đến thời điểm hiện tại thì đã xuất hiện rất nhiều các kỹ thuật prompting biến thể khác nhau tuy nhiên thì 3 kỹ thuật nêu ở trên vẫn được sử dụng rất phổ biến trong các ứng dụng thực tế.

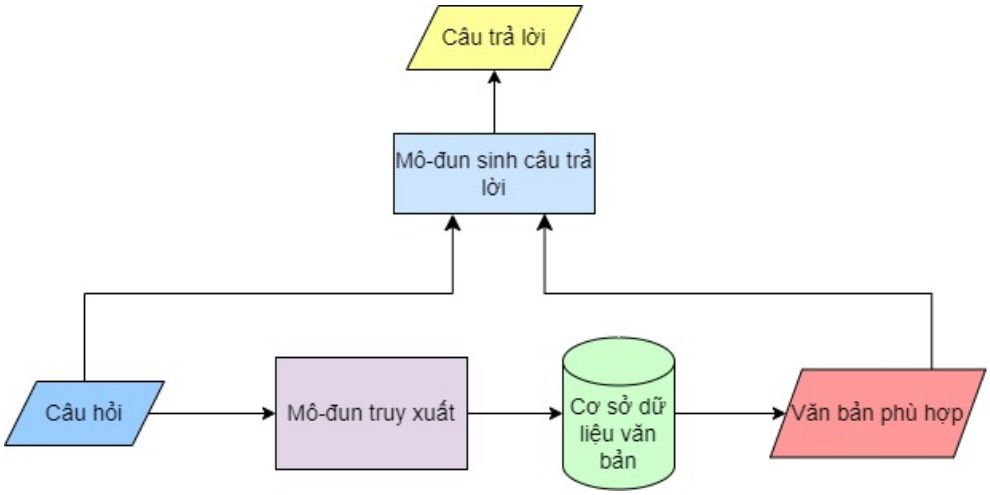
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

Trong chương này em sẽ đề xuất các giải pháp để xây dựng hệ thống hỏi đáp cho các bạn sinh viên Đại học Bách Khoa Hà Nội sao cho hệ thống đạt được độ chính xác cao và vẫn phải đáp ứng được yêu cầu về thời gian. Trong phần 3.1 em sẽ trình bày về tổng quan giải pháp bao gồm việc chọn mô hình backbone, tập dữ liệu sử dụng, các thành phần trong mô-đun truy xuất và cách kết hợp với mô hình BM25 để tăng hiệu quả truy xuất. Trong phần 3.2 và 3.3 em sẽ trình bày về 2 thành phần chính trong mô-đun truy xuất là retriever và reranker. Trong phần 3.4 em sẽ trình bày về giải pháp cho mô-đun sinh câu trả lời.

3.1 Tổng quan giải pháp

Hệ thống của em phát triển dựa trên hai mô-đun chính là: truy xuất và sinh câu trả lời và tổng quan hệ thống được em trình bày trong hình 3.1

Đầu tiên câu hỏi đầu vào sẽ được cho qua mô-đun truy xuất kết hợp với cơ sở



Hình 3.1: Tổng quan hệ thống hỏi đáp

dữ liệu chứa văn bản để chọn ra top k văn bản phù hợp nhất với câu hỏi (k là một siêu tham số có thể điều chỉnh). Sau đó cả câu hỏi và các văn bản được lấy ra sẽ được cho qua mô-đun sinh câu trả lời để tạo ra câu trả lời hoàn chỉnh.

3.1.1 Backbone sử dụng

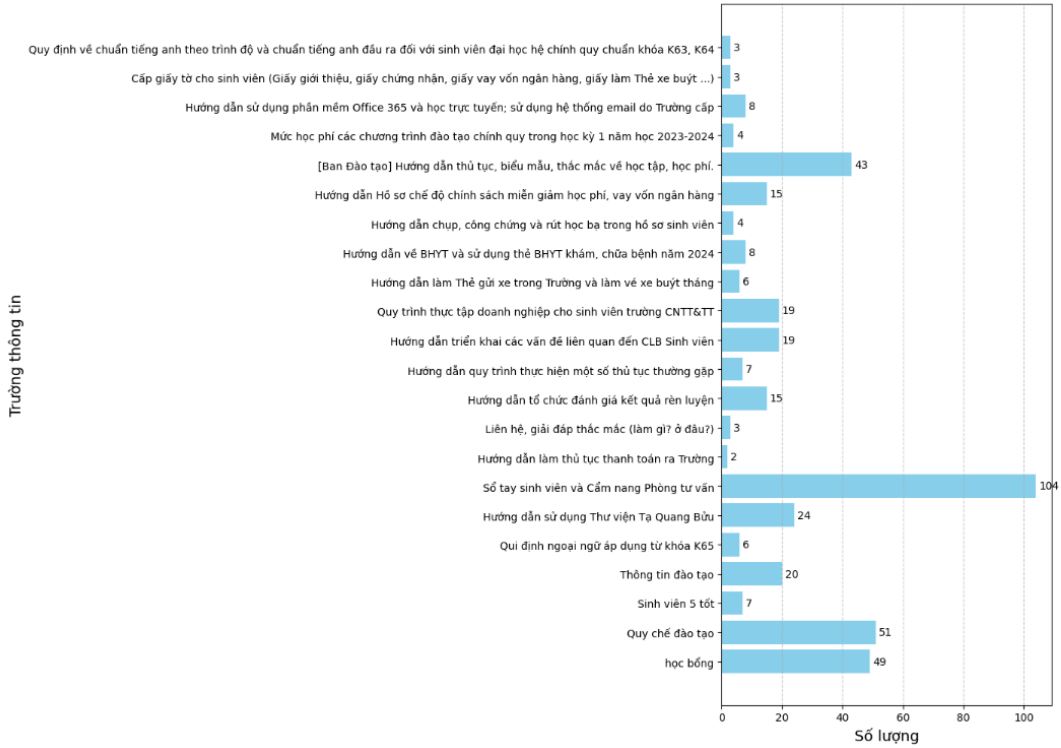
Cả hai kiến trúc bi-encoder và cross-encoder đều được em huấn luyện với mô hình backbone là PhoBERT-large [24]. PhoBERT-large có kiến trúc là mô hình RoBERTa [25] được tiền huấn luyện trên 20GB dữ liệu trên Wikipedia và dữ liệu tin tức bằng tiếng việt. PhoBERT-large có tổng cộng 370 triệu tham số và hỗ trợ tối đa đầu vào là 256 token. PhoBERT-large được VinAI cho ra mắt năm 2020 và đến nay thì nó vẫn được sử dụng vô cùng phổ biến trong cộng đồng xử lý ngôn ngữ

tự nhiên cho tiếng việt.

Đối với phương pháp BGE M3-Embedding em sử dụng mô hình backbone là BAAI/bge-m3 trên Hugging Face. Mô hình này sử dụng bộ mã hóa văn bản là XLM-RoBERTa, ban đầu mô hình BAAI/bge-m3 được huấn luyện không giám sát trên tập dữ liệu lớn và sau đó được tinh chỉnh trên cả tập dữ liệu có nhãn và tập dữ liệu tổng hợp. Mô hình BAAI/bge-m3 đang hỗ trợ hơn 100 ngôn ngữ khác nhau.

3.1.2 Tập dữ liệu

Dữ liệu được em thu thập trên các trang thông tin chính thống của Đại học Bách Khoa Hà Nội như là ctsv.hust.edu.vn, ctt.hust.edu.vn, https://soict.hust.edu.vn/, ... Dữ liệu sẽ bao hàm nhiều chủ đề thông tin khác nhau như là về chương trình đào tạo, học bổng, đánh giá điểm rèn luyện,... Sau cùng dữ liệu em thu thập gồm 22 chủ đề khác nhau và tổng số lượng văn bản là 420. Phân bố số lượng văn bản cho từng chủ đề được em biểu diễn trong hình 3.2.



Hình 3.2: Phân bổ số lượng văn bản cho từng chủ đề

Với mỗi một chủ đề sẽ bao gồm nhiều nội dung khác nhau, mỗi nỗi dung sẽ được đặc trưng bởi hai thông tin là (i) tiêu đề và (ii) mô tả chi tiết cho nội dung đó. Trong bảng 3.1 em có minh họa về một nội dung nằm trong chủ đề về "Quy chế đào tạo", nội dung này có tiêu đề là "Điểm trung bình toàn khóa và hạng tốt nghiệp đại học" và có mô tả chi tiết ở trong bảng.

Chủ đề	Quy chế đào tạo
Tiêu đề	Điểm trung bình toàn khóa và hạng tốt nghiệp đại học
Mô tả chi tiết	1. Điểm trung bình toàn khóa là điểm trung bình tích lũy toàn khóa tính theo các học phần đăng ký tốt nghiệp. 2. Hạng tốt nghiệp được xếp dựa trên điểm trung bình toàn khóa như xếp loại học lực quy định tại khoản 6 Điều 12 của Quy chế này, riêng trong các trường hợp sau đây thì hạng tốt nghiệp của những sinh viên có điểm trung bình toàn khóa xếp loại giỏi trở lên sẽ bị giảm một mức: a) Khối lượng của các học phần phải học lại vượt quá 5% so với tổng số TC quy định cho toàn CTĐT. Quy định này không xét tới số TC của các học phần học cải thiện điểm. b) Sinh viên bị Hội đồng kỷ luật của ĐHBK Hà Nội đề nghị áp dụng mức kỷ luật từ cảnh cáo trở lên.

Bảng 3.1: Minh họa chi tiết về một nội dung trong một chủ đề

Bởi vì mô hình backbone PhoBERT của các mô hình bi-encoder chỉ hỗ trợ được tối đa 256 token đầu vào nên em cũng quan tâm đến độ dài của các văn bản. Bảng 3.3 phân tích phân bổ về độ dài các văn bản trong tập dữ liệu. Từ phân tích có thể thấy tỉ lệ độ dài văn bản nhỏ hơn 256 chiếm đến gần 84% và chỉ có khoảng 16% là vượt ngưỡng 256.

Để có thể tinh chỉnh các mô hình cho tác vụ truy xuất thông tin thì tập dữ liệu truy xuất có gán nhãn là vô cùng cần thiết. Vậy nên em đã xây dựng ra bộ dữ liệu truy xuất này, bao gồm 1521 cặp (câu hỏi, tập văn bản chứa câu trả lời). Bộ dữ liệu này được em tạo ra bằng cách sử dụng GPT4 của công ty OpenAI, với mỗi văn bản em sẽ cho GPT4 sinh ra các câu hỏi tương ứng với văn bản đó. Sau đó tất cả các câu hỏi đều được em rà soát và điều chỉnh lại để đảm bảo về độ tin cậy của câu hỏi.

Bảng 3.2 đưa ra một ví dụ về một mẫu trong tập dữ liệu truy xuất. Trong đó mỗi câu hỏi sẽ có tập các văn văn bản phù hợp tương ứng, và mỗi một văn bản sẽ được xác định thông qua hai trường là "Chủ đề" và "Tiêu đề" của văn bản.

Toàn bộ 1521 cặp trên em sẽ chia ra 1211 cặp cho tập train và 310 cặp cho tập evaluate. Để đánh giá một cách chuẩn xác thì tập test được em lấy từ các bài kiểm tra quy chế của Đại học Bách Khoa Hà Nội từ kỳ 2022.1 cho đến hết kỳ 2023.2. Sau khi loại bỏ các câu trùng nhau và dựa vào đáp án đã cho em tiến hành gán nhãn cho từng câu hỏi. Tập test cuối cùng mà em thu được là 300 cặp (câu hỏi , tập các văn bản chứa câu trả lời). Và để đánh giá cho mô-đun sinh câu trả lời cũng như là

Câu hỏi	Đối với các học phần đồ án thì bao nhiêu điểm được tính là điểm liệt?
Văn bản tương ứng	{ "Chủ đề": "Thông tin đào tạo" , "Tiêu đề": "Đánh giá kết quả học tập" }

Bảng 3.2: Ví dụ về một mẫu dữ liệu trong tập dữ liệu truy xuất

toàn bộ pipeline thì với mỗi câu hỏi trong tập test em còn bổ sung cả câu trả lời hoàn chỉnh cho câu hỏi đó.

Độ dài	**Số lượng**	**Tỉ lệ**
<100	206	49,05%
101-256	152	36,19%
257+	62	14,76%

Bảng 3.3: Phân bố độ dài trong tập dữ liệu văn bản

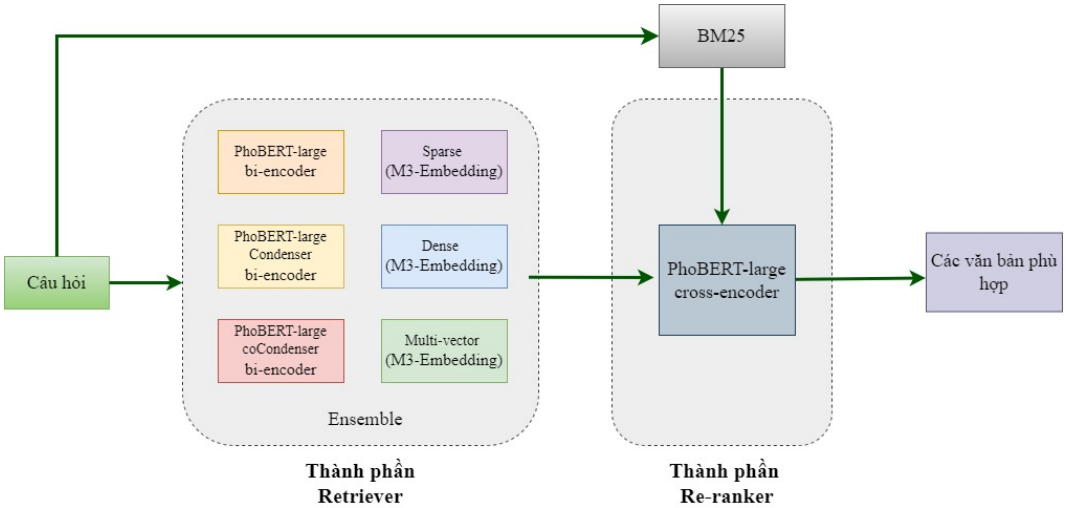
3.1.3 Các thành phần của mô-đun truy xuất

Trong mô-đun này em sẽ chia thành 2 thành phần nhỏ hơn là (i) Retriever và (ii) Reranker . Luồng hoạt động của mô-đun truy xuất này được em thể hiện trong hình 3.3.

Trong thành phần Retriever thì kết quả của 3 mô hình bi-encoder là PhoBERT-large, PhoBERT-large Condenser và PhoBERT-large coCondenser cùng với 3 chức năng truy xuất của M3-Embedding là truy xuất dày đặc, truy xuất thưa thớt và truy xuất đa véc-tơ sẽ được ensemble lại với nhau theo kỹ thuật Weighted Averaging (lấy trung bình có trọng số).

Từ câu hỏi của các bạn sinh viên, thành phần Retriever sẽ truy xuất ra top k văn bản phù hợp nhất với câu hỏi dựa trên sự tương đồng. Sau đó thành phần Re-ranker tận dụng thế mạnh của mình là độ chính xác cao để xếp hạng lại top k văn bản này.

Lý do em phải chia thành 2 thành phần nhỏ hơn thay vì chỉ sử dụng thành phần Re-ranking để đạt được kết quả truy xuất cao bởi vì đối với kiến trúc cross-encoder của thành phần Re-ranker khi thực hiện suy luận sẽ phải đưa từng cặp (câu hỏi, văn bản) để tính ra độ tương đồng. Điều này sẽ gây ra thời gian suy luận sẽ rất lâu khi mà số văn bản trong tập cơ sở dữ liệu nhiều. Còn các mô hình trong Retriever thì việc mã hóa các văn bản đều độc lập nên có thể dễ dàng lưu lại vec-tơ nhúng của các đoạn văn này. Vậy nên khi câu hỏi cho qua Retrever thì nó chỉ phải mã hóa câu hỏi đó thành vec-tơ, sau đó có thể sử dụng vec-tơ này để tính độ tương đồng với



Hình 3.3: Luồng hoạt động của mô-đun truy xuất

các vec-tơ của các đoạn văn đã được lưu sẵn trong cơ sở dữ liệu. Số lượng văn bản sau Retriever đã giảm đi rất nhiều, sau đó có thể tận dụng Reranker để xếp hạng lại kết quả của các văn bản này để có được độ chính xác cao hơn.

3.1.4 Kết hợp với BM25

Trong quá trình chạy suy luận thì em có phân tích và thấy được một số câu hỏi bị truy xuất sai rơi vào trường hợp câu hỏi không nặng về mặt ngữ nghĩa như là: "Địa chỉ Email A là của phòng ban nào?". Những trường hợp như thế này em thấy nếu như tận dụng khả năng so khớp từ vựng của BM25 thì sẽ giải quyết được. Vậy nên em đã kết hợp cả BM25 vào mô-đun truy xuất của mình và khi này điểm cho độ tương đồng sẽ được tính như sau:

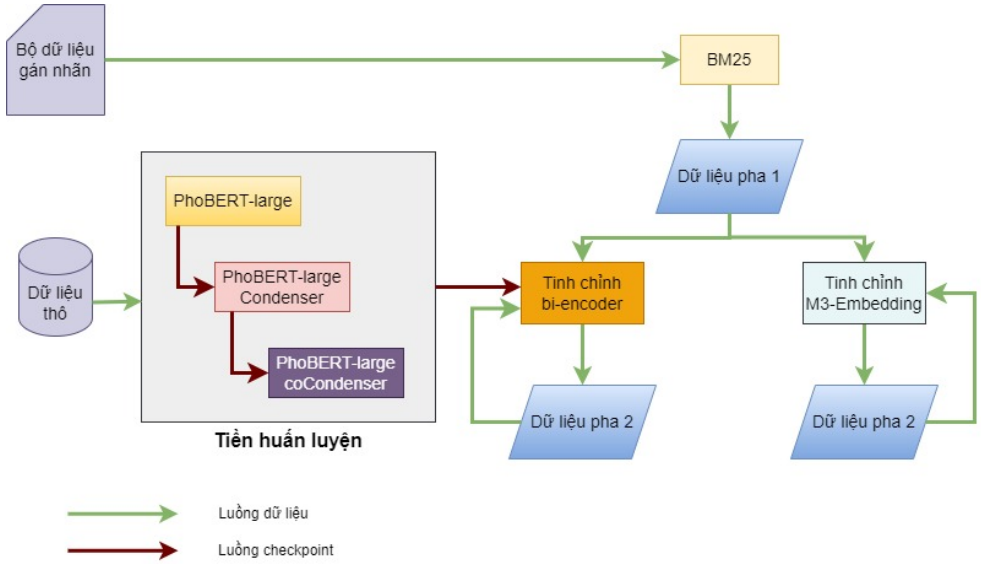
$$\text{score}_{\text{truy xuất}} = (\text{score}_{\text{retriever}} + \text{score}_{\text{reranker}}) \cdot \text{score}_{\text{BM25}} \tag{3.1}$$

Trong đó :

- $\text{score}_{\text{truy xuất}}$: điểm tương đồng của toàn bộ mô-đun truy xuất
- $\text{score}_{\text{retriever}}$: điểm tương đồng của thành phần retriever
- $\text{score}_{\text{reranker}}$: điểm tương đồng của thành phần reranker
- $\text{score}_{\text{bm25}}$: điểm tương đồng của BM25

3.2 Thành phần Retriever

Toàn bộ quy trình huấn luyện các mô hình của thành phần Retriever được em thể hiện trong hình 3.4. Trong đó sẽ bao gồm việc tiền huấn luyện, tinh chỉnh các mô hình bi-encoder và tinh chỉnh mô hình M3-Embedding.



Hình 3.4: Luồng huấn luyện các mô hình trong thành phần Retriever

3.2.1 Quá trình tiền huấn luyện

Ban đầu, mô hình PhoBERT-large được em tiền huấn luyện với tác vụ Masked Language Model trên toàn bộ dữ liệu văn bản thô. Mô hình PhoBERT-large sau tiền huấn luyện được em sử dụng làm mô hình backbone cho kiến trúc PhoBERT-large Condenser với trọng số của các lớp Condenser head được khởi tạo ngẫu nhiên. PhoBERT-large Condenser sau đó sẽ lại được tiền huấn luyện trên tập dữ liệu văn bản thô với tác vụ Masked Language Model. Mong muốn sau quá trình tiền huấn luyện thì việc học biểu diễn cho token CLS được hiệu quả hơn.

Cuối cùng PhoBERT-large coCondenser với kiến trúc được kế thừa từ PhoBERT-large Condenser được khởi tạo trọng số từ mô hình PhoBERT-large Condenser được tiền huấn luyện ở trên. PhoBERT-large coCondenser được tiền huấn luyện tiếp trên tập dữ liệu sau khi đã qua bước tiền xử lý để học được biểu diễn tương phản không giám sát ở cấp độ văn bản.

3.2.2 Quá trình tinh chỉnh

a, Tinh chỉnh với dữ liệu truy xuất pha 1

Bộ dữ liệu truy xuất pha 1 được em khai phá bằng bằng mô hình BM25. Ở đây, với mỗi câu truy vấn em sẽ lấy ra 20 văn bản tương đồng nhất dựa trên điểm số của BM25. Tiếp theo các văn bản này sẽ đựa chunking thành các đoạn văn bản nhỏ hơn, những văn bản nào nằm trong danh sách tương ứng được gán nhãn với câu truy vấn trong bộ dữ liệu thu thập thì tất cả các đoạn nhỏ sau khi chunking đều được gán là các mẫu dương và tất cả các văn bản còn lại trong 20 văn bản được lấy ra sẽ là các mẫu âm khó. Tổng số cặp mẫu em tạo ra bao gồm 39728 cặp (câu hỏi , văn bản). Tổng số cặp này sẽ được em chia thành 30000 cặp cho tập train và 9728

cặp cho tập evaluate.

Cả 3 mô hình PhoBERT-large, PhoBERT-large Condenser và PhoBERT-large coCondenser sau khi tiền huấn luyện sẽ lần được được tinh chỉnh trên bộ dữ liệu truy xuất pha 1 . Các mô hình này đều được huấn luyện với hàm mất mát tương phản để học được không gian biểu diễn. Để đánh giá các mô hình này thì em dựa trên việc phân loại đầu ra với 1 thể hiện các cặp mẫu dương và 0 là các cặp mẫu âm. Bên cạnh đó em cũng tận dụng bộ dữ liệu truy xuất pha 1 này để tinh chỉnh cho mô hình M3-Embedding.

b, Tinh chỉnh với dữ liệu truy xuất pha 2

Dữ liệu pha 1 được khai thác bằng BM25 vậy nên các mẫu âm khó pha 1 có thiên hướng về so khớp từ vựng. Vậy để tạo các mẫu âm khó về mặt ngữ nghĩa thì em sẽ tiền hành thực hiện suy luận từ các mô hình đã tinh chỉnh ở pha 1.

Việc khai phá các mẫu âm pha 2 của 3 mô hình bi-encoder đều giống với các bước làm ở pha 1, chỉ có 1 chút khác biệt đối với mô hình M3-Embedding là mô hình này sẽ huấn luyện đồng thời cả 3 chức năng truy xuất là Dense, Sparse và Multi-vector retrieval vậy nên em sẽ đựa trên điểm trung bình của 3 chức năng truy xuất này để tạo ra các mẫu âm pha 2 cho huấn luyện M3-Embedding. Số lượng cặp các tập dữ liệu truy xuất pha 2 được em thống kê ở bảng 3.4.

Mô hình	**Tập train**	**Tập test**	**Tổng**
PhoBert-large pha 1	19000	5346	24346
PhoBert-large Condenser pha 1	19000	5355	24355
PhoBert-large coCondenser pha 1	19000	5334	24334
M3-Embedding pha 1	19000	4978	23978

Bảng 3.4: Thống kê số cặp các bộ dữ liệu truy xuất pha 2

3.3 Thành phần Re-ranker

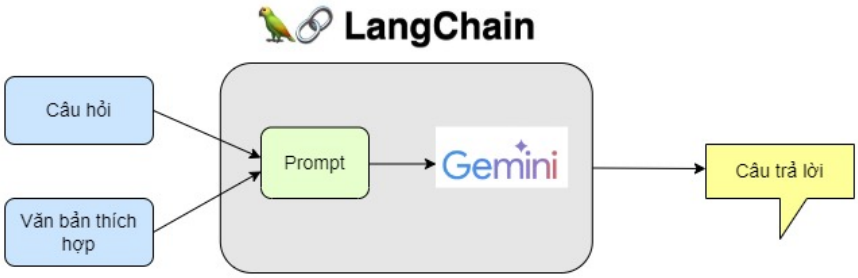
Mục đích của Re-ranker sẽ là xếp hạng lại kết quả từ Retriever với mục đích để tăng độ chính xác của việc xếp hạng. Để tạo thành tập dữ liệu huấn luyện cho mô hình cross-encoder của Re-ranker em tiến hành khai phá tiếp các mẫu âm khó trên 3 mô hình là PhoBERT-large bi-encoder, PhoBERT-large Condenser bi-encoder, PhoBERT-large coCondenser bi-encoder và mô hình M3-Embedding đã tinh chỉnh pha 2.

Mô hình backbone PhoBERT-large của mô hình cross-encoder đã được tiền huấn luyện trên tập dữ liệu văn bản thô, điều này sẽ giúp cho mô hình có thể

hiểu và thích ứng với các quy định, quy chế, cũng như là các miền thông tin khác của Đại học Bách Khoa Hà Nội. Lúc này mô hình cross-encoder chỉ là việc phân loại nhị phân với đầu ra chỉ là 0 và 1 .

3.4 Mô-đun sinh câu trả lời

Luồng hoạt động của mô-đun sinh câu trả lời được em trình bày trong hình 3.5. Ban đầu LangChain tiếp nhận câu hỏi và các đoạn đoạn văn bản thích hợp, sau đó LangChain tạo ra một prompt bằng cách kết hợp câu hỏi và các đoạn văn bản này để tạo ra ngữ cảnh cần thiết cho Gemini. Sau khi prompt được gửi tới Gemini thì mô hình này sẽ phân tích ngữ cảnh và thông tin trong prompt, sử dụng khả năng suy luận của mình để sinh ra câu trả lời chính xác và tự nhiên.



Hình 3.5: Luồng hoạt động của mô-đun sinh câu trả lời

Các thành phần trong mô-đun sinh câu trả lời:

- **LangChain** là một framework mạnh mẽ được thiết kế để tích hợp và quản lý các mô hình ngôn ngữ lớn. Nó cung cấp các công cụ và giao diện cần thiết để dễ dàng sử dụng các mô hình này trong các ứng dụng thực tế. LangChain giúp điều phối các tác vụ liên quan đến xử lý ngôn ngữ tự nhiên, từ việc tạo ra các prompt cho đến thập phản hồi từ các mô hình ngôn ngữ lớn.
- **Prompt** là một thành phần quan trọng trong việc tương tác với các mô hình ngôn ngữ lớn và được quản lý bởi LangChain. Prompt đóng vai trò ngữ cảnh được cung cấp cho mô hình để hướng dẫn mô hình tạo ra câu trả lời. Trong mô-đun này, prompt được LangChain tạo ra từ câu hỏi và đoạn văn bản liên quan. LangChain định dạng prompt này để cung cấp thông tin đầy đủ và cần thiết, giúp mô hình hiểu rõ ngữ cảnh và sinh ra câu trả lời phù hợp. LangChain cung cấp các mẫu định dạng prompt chuẩn hóa để đảm bảo rằng thông tin được truyền đạt một cách hiệu quả nhất tới mô hình ngôn ngữ lớn.
- **Gemini** là mô hình ngôn ngữ lớn được em lựa chọn trong mô-đun này bởi khả năng hiểu và tạo ngữ cảnh một cách tự nhiên và chính xác. Gemini còn

hỗ trợ cho đa ngôn ngữ trong đó có tiếng việt. Phiên bản Gemini em sử dụng là Gemini 1.5 Pro sở hữu cửa sổ ngữ cảnh lên tới 128.000 token và hiện nay Gemini đã được tích hợp sẵn vào LangChain.

CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM

Trong chương này, em sẽ trình bày các kết quả thực nghiệm để so sánh và đưa ra cái nhìn tổng thể về các phương pháp mà em đã đề cập và đề xuất. Trong phần đầu tiên em sẽ trình bày về việc thiết lập các tham số đánh giá và các tiêu chí đánh giá cho các thực nghiệm (phần 4.1). Trong phần tiếp theo em sẽ trình bày về việc lựa chọn các baseline để so sánh cũng như là đưa ra các phương án thử nghiệm (phần 4.2) và các phần còn lại em sẽ trình bày chi tiết về từng thử nghiệm.

4.1 Phương pháp đánh giá

4.1.1 Các tham số đánh giá

4.1.2 Các tiêu chí đánh giá

Em mong muốn hệ thống hỏi đáp của mình có thể ứng dụng cho tất cả các bạn sinh viên tại Đại học Bách Khoa Hà Nội,vậy nên độ chính cao là tiêu chí mà em đặt lên hàng đầu. Bên cạnh đó thì tốc độ truy xuất và trả lời câu hỏi cũng phải nhanh để có thể đáp ứng được nhu cầu sử dụng trong thực tiễn.

Các độ đo được em sử dụng để đánh giá như sau:

1. Đối với mô-đun Retrieval, em sử dụng phương thức để đánh giá đó là Acc@k và Recall@k. Trong đó em chọn k lần lượt là 3,5 và 10. Đối với phương thức đánh giá Acc@k thì độ chính xác đạt giá trị tuyệt đối khi chỉ cần có một văn bản trả lời đúng trong top k văn bản được truy xuất. Còn đối với Recall@k, nếu trong top k văn bản được truy xuất chứa đủ tất cả các văn bản tương ứng với câu truy vấn thì được tính là chính xác .Cả hai phương thức đánh giá trên đều có đơn vị là %

2. Để đánh giá về vấn đề thời gian truy xuất của từng phương pháp em lựa chọn thời gian tính bằng giây.

Dữ liệu đánh giá

Em sử dụng tập test mà em đã đề cập đến ở phần Phương pháp đề xuất. Tập dữ liệu bao gồm 300 cặp (câu hỏi và văn bản trả lời tương ứng)

4.2 Thử nghiệm

4.2.1 Các baseline sử dụng để so sánh

Hai mô hình em sử dụng để so sánh là **bkai-foundation-models/vietnamese-bi-encoder** và **keepitreal/vietnamese-sbert** trên Hugging Face. Cả hai mô hình em lựa chọn này đều sử dụng kiến trúc là bi-encoder và hầu hết đều được mọi người sử dụng trong các tác vụ lên quan trên truy xuất thông tin cho tiếng viêt. Dưới đây em sẽ trình bày về cách các mô hình baseline được huấn luyện.

Đầu tiên em sẽ trình bày về mô hình vietnamese-bi-encoder. Mô hình này được huấn luyện trên các tập dữ liệu cho tiếng việt trong đó có tập dữ liệu về pháp luật của cuộc thi Zalo AI Challenge năm 2021. Backbone của mô hình này là phobert-base-v2. Hàm mất mát được sử dụng để huấn luyện mô hình trên là Multiple Negatives Ranking Loss với kích thước lô là 32, trong đó số mẫu âm cho mỗi một câu truy vấn là 20 và được huấn luyện trong 15 epoch.

Còn với vietnamese-sbert,hiện tại thì mô hình này chưa công bố các tập dữ liệu mà nó đã huấn luyện. Hàm mất mát sử dụng trong mô hình này là Cosine Similarity Loss .

4.2.2 Cấu hình các tham số

Tất cả các thử nghiệm của em trong quá trình thực hiện đồ án bao gồm việc tiền huấn luyện, huấn luyện, tinh chỉnh và suy luận đều được thực hiện trên 1 GPU P100 16GB VRAM.

Trong BM25 thì 2 tham số k1 và b được em chọn với giá trị lần lượt là 0.4 và 0.6.

Mô hình PhoBERT-large được tiền huấn luyện với task vụ Masked Language Model, trong đó kích thước lô là 16, tỉ lệ che (mask) là 15% và được huấn luyện trong 30 epoch.

Mô hình PhoBERT-large Condenser được tiền huấn luyện trong đó tỉ lệ che (mask) trong Masked Language Model là 15%, kích thước lô là 16 và được huấn luyện trong 20 epoch.

Mô hình PhoBERT-large coCondenser được tiền huấn luyện với kích thước lô con là 16 và được huấn luyện trong 20 epoch.

Khi tinh chỉnh các mô hình bi-encoder thì em sử dụng độ chính xác float16 và kích thước lô là 16. Số epoch được huấn luyện trong pha 1 là 10 epoch và trong pha 2 là 8 epoch.

Mô hình M3-Embedding được tinh chỉnh với kích thước lô bằng 2, độ dài tối đa của câu truy vấn và văn bản lần lượt là 128 và 256, sử dụng độ chính xác là float16 và được huấn luyện trong 10 epoch trong pha 1 và 8 epoch trong pha 2.

Khi tinh chỉnh cross-encoder em thử nghiệm với kích thước lô bằng 16 và số epoch là 10.

4.2.3 Các phương án thử nghiệm

Tất cả các thử nghiệm đều được em đo duy nhất một lần. Các phương án thử nghiệm của em bao gồm:

1. Tinh chỉnh mô hình PhoBERT-large đã được tiền huấn luyện trong pha 1 đối với các hàm mất mát bao gồm: Softmax Loss, Multiple Negatives Ranking

Loss, Contrastive Loss và Online Contrastive Loss. Thử nghiệm này nhằm tìm ra hàm mất mát phù hợp nhất cho việc tinh chỉnh các mô hình bi-encoder trên bộ dữ liệu truy xuất.

2. So sánh kết quả truy xuất của mô hình bi-encoder pha 1 khi tinh chỉnh với backbone chưa được tiền huấn luyện và đã được huấn luyện thêm trên bộ dữ liệu văn bản thô. Thử nghiệm này có mục đích là kiểm tra tính cần thiết của việc tiền huấn luyện thêm để giúp mô hình thích ứng với miền văn bản thông tin của Đại học Bách Khoa Hà Nội.

3. So sánh kết quả truy xuất của các mô hình bi-encoder pha 1 khi tinh chỉnh với backbone được tiền huấn luyện với các phương pháp khác nhau bao gồm: kiến trúc BERT tiêu chuẩn, kiến trúc Condenser và coCondenser. Thử nghiệm này nhằm mục đích kiểm chứng tính vượt trội với việc học biểu diễn tương phản không giám sát ở cấp độ kho văn bản giúp warm up không gian biểu diễn khi tiền huấn luyện với kiến trúc coCondenser .

4. So sánh kết quả truy xuất của các mô hình bi-encoder pha 1 trong hai trường hợp là tinh chỉnh với dữ liệu đã được chunking và không được chunking. Thử nghiệm này nhằm mục đích kiểm chứng tính hiệu quả của việc chunking dữ liệu, giúp giải quyết được những trường hợp mà các văn bản cần truy xuất có kích thước vượt quá kích thước tối đa mà mô hình backbone của bi-encoder hỗ trợ.

5. So sánh kết quả truy xuất khi tinh chỉnh các mô hình bi-encoder trong pha 1 và pha 2 với dữ liệu đã được chunking. Thử nghiệm này chứng minh sự cần thiết của việc khai thác các mẫu âm khó trong pha 2.

6. So sánh kết quả của từng chức năng truy xuất của mô hình M3-Embedidng với kết quả các mô hình bi-encoder pha 1 khi tinh chỉnh mô hình M3-Embedding với dữ liệu pha 1. Trong thử nghiệm này em muốn đánh giá tính hiệu quả của kĩ thuật tự chắt lọc tri thức (Self-Knowledge Distillation) trong việc huấn luyện đồng thời 3 chức năng truy xuất là truy xuất thưa thớt, truy xuất dày đặc và truy xuất đa véc-tơ nhằm nâng cao hiệu quả của từng chức năng này.

7. So sánh kết quả truy xuất khi tinh chỉnh mô hình M3-Embedding với dữ liệu pha 1 và pha 2. Trong thử nghiệm này em muốn đánh giá xem việc khai thác các mẫu âm pha 2 có hiệu quả để tinh chỉnh cho mô hình M3-Embedding hay không.

8. So sánh kết quả truy xuất của các mô hình bi-encoder và ba chức năng truy xuất của mô hình M3-Embedding ở pha 2 với các mô hình baseline mà em đã

lựa chọn ở trên.

9. So sánh kết quả truy xuất trong các trường hợp bao gồm 3 mô hình bi-encoder, 3 chức năng truy xuất của mô hình M3-Embedding, tiếp theo là ensemble 3 mô hình bi-encoder cùng với 3 chức năng truy xuất của M3-Embedding, tiếp đến là bổ sung lần lượt mô hình cross-encoder và BM25 vào kết quả đã được ensemble trước đó. Cuối cùng là việc bổ sung cả mô hình cross-encoder và BM25 vào kết quả ensemble 3 mô hình bi-encoder với 3 chức năng truy xuất của M3-Embedding. Thử nghiệm này nhằm mục đích kiểm chứng tính hiệu quả của việc ensemble kết quả nhiều mô hình lại với nhau cũng như là sự cần thiết của việc xếp hạng lại của mô mình cross-encoder và khả năng so khớp từ vựng của BM25.

10. So sánh thời gian truy xuất trung bình trên một câu truy vấn với các mô hình trong mô-đun truy xuất. Trong thử nghiệm này em muốn đánh giá tốc độ truy xuất của từng loại mô hình cũng như về độ khả thi của các mô hình này trong triển khai hệ thống thực tế.

4.3 Thử nghiệm tinh chỉnh mô hình bi-encoder với từng hàm mất mát khác nhau

Sau khi tiến hành tiền huấn luyện bổ sung cho mô hình PhoBERT-large trên dữ liệu văn bản thô, em đã thử nghiệm tinh chỉnh mô hình này bằng cách sử dụng các hàm mất mát khác nhau bao gồm: Softmax Loss, Multiple Negatives Ranking Loss, Contrastive Loss và Online Contrastive Loss. Với mỗi hàm mất mát em đều thực hiện đánh giá kết quả và kết quả thử nghiệm được em trình bày trong bảng 4.1 và bảng 4.2.

Hàm mất mát	Acc@3	Acc@5	Acc@10
Softmax Loss	21,67	27,33	35,67
Multiple Negatives Ranking Loss	59,67	69,33	78,33
Contrastive Loss	85,67	87,67	92,1
Online Contrastive Loss	**87,33**	**89,66**	**92,33**

Bảng 4.1: Kết quả độ chính xác Acc@k (%) khi tinh chỉnh pha 1 mô hình PhoBERT-large bi-encoder với từng hàm mất mát

Từ kết quả của hai bảng so sánh có thể dễ dàng thấy được kết quả khi tinh chỉnh với hai hàm mất mất Online Contrastive Loss và Contrastive Loss cho kết quả vượt trội hơn so với các hàm mất mát còn lại. Điều này cho thấy được tính hiệu quả của việc học biểu diễn tương phản. Bên cạnh đó, kết quả của hàm mất mát Online

Hàm mất mát	Recall@3	Recall@5	Recall@10
Softmax Loss	21,05	27,33	35,33
Multiple Negatives Ranking Loss	59,05	68,55	78,00
Contrastive Loss	85,33	87,33	92,1
Online Contrastive Loss	**87,00**	**89,5**	**92,33**

Bảng 4.2: Kết quả Recall@k (%) khi tinh chỉnh pha 1 mô hình PhoBERT-large bi-encoder với từng hàm mất mát

Contrastive Loss tốt hơn so với hàm mất mát Contrastive Loss có thể được giải thích bởi Online Contrastive Loss tập trung vào học biểu diễn của các mẫu dương và các mẫu âm khó. Điều này đã giúp mô hình học được không gian biểu diễn tốt hơn so với khi sử dụng hàm mất mát Contrastive Loss.

Đứng sau kết quả hai hàm mất mát ở trên đó là hàm mất mát Multiple Negatives Ranking Loss. Kết quả này thấp hơn so với hai hàm mất mát trước đó cũng xuất phát từ chính cách huấn luyện của hàm mất mát này. Đầu vào của hàm mất mát này là các cặp (truy vấn, văn bản) và hàm này sử dụng tất cả các văn bản của các truy vấn khác trong một lô làm các mẫu âm và lấy văn bản tương ứng với truy vấn đó làm mẫu dương. Tuy nhiên, nếu trong cùng một lô có các câu truy vấn liên quan đến cùng một văn bản, điều này sẽ tạo ra các mẫu âm sai, dẫn đến kết quả huấn luyện không chính xác.

Cho kết quả thấp nhất trên cả hai phương pháp đánh giá đó là khi tinh chỉnh với hàm mất mát Softmax Loss. Điều này cho thấy hàm mất mát này không thực sự hiệu quả trong việc học không gian biểu diễn và càng chứng minh được tầm quan trọng của học biểu diễn tương phản cho mô-đun truy xuất.

4.4 So sánh kết quả khi tinh chỉnh pha 1 với backbone chưa được tiền huấn luyện và đã được tiền huấn luyện thêm

Trong thử nghiệm này em tiến hành tinh chỉnh mô hình bi-encoder với dữ liệu pha 1 trong hai trường hợp: mô hình PhoBERT-large chưa được tiền huấn luyện bổ sung và đã được tiền huấn luyện bổ sung với tập dữ liệu văn bản thô. Với mỗi trường hợp em đều thực hiện đánh giá và kết quả của thử nghiệm được em trình bày trong bảng 4.3

K	Mô hình	Acc	Recall
3	PhoBERT-large bi-encoder pha 1 với backbone chưa được tiền huấn luyện thêm	79,33	78,66
	PhoBERT-large bi-encoder pha 1 với backbone đã được tiền huấn luyện thêm	87,33	87
5	PhoBERT-large bi-encoder pha 1 với backbone chưa được tiền huấn luyện thêm	80,67	80,33
	PhoBERT-large bi-encoder pha 1 với backbone đã được tiền huấn luyện thêm	88,33	88,33
10	PhoBERT-large bi-encoder pha 1 với backbone chưa được tiền huấn luyện thêm	84,33	84,33
	PhoBERT-large bi-encoder pha 1 với backbone đã được tiền huấn luyện thêm	92,33	92,33

Bảng 4.3: Kết quả độ chính xác Acc(%) và Recall(%) khi tinh chỉnh mô hình bi-encoder với backbone chưa được tiền huấn luyện thêm và đã được tiền huấn luyện thêm

Từ kết quả bảng 4.3, việc tinh chỉnh PhoBERT-large với backbone đã được tiền huấn luyện thêm trên tập dữ liệu thô cho thấy kết quả vượt trội so với backbone chưa được tiền huấn luyện thêm trong cả hai phương thức đánh giá. Điều này chứng tỏ sự cần thiết của việc tiền huấn luyện bổ sung, giúp mô hình thích ứng tốt hơn với miền thông tin liên quan đến Đại học Bách Khoa Hà Nội.

4.5 Tinh chỉnh mô hình bi-encoder với backbone được tiền huấn luyện với các phương pháp khác nhau

Đầu tiên, em tiến hành tiền huấn luyện mô hình PhoBERT-large với tác vụ Masked Language Model trên tập dữ liệu văn bản thô. Tiếp theo, mô hình này được tiếp tục tiền huấn luyện với kiến trúc Condenser. Cuối cùng, mô hình sau khi tiền huấn luyện với Condenser được tiếp tục tiền huấn luyện với coCondenser. Cả ba mô hình sau khi tiền huấn luyện sẽ được sử dụng để tinh chỉnh mô hình bi-encoder với dữ liệu pha 1. Toàn bộ kết quả của thử nghiệm này được em trình bày trong bảng 4.4

Từ kết quả của bảng so sánh có thể thấy được khi tiền huấn luyện với coCondenser sẽ cho kết quả tinh chỉnh là tốt nhất. Điều này cho thấy sự hiệu quả của việc học biểu diễn tương phản không giám sát ở cấp độ kho văn bản, đã giúp warmup không gian nhúng và giúp nâng cao chất lượng của việc tinh chỉnh ở bước sau.

Cho kết quả thấp nhất là kiến trúc Condenser, đặc biệt khi k = 3 cho kết quả thấp hơn hẳn so với các kết quả còn lại. Nguyên nhân chính dẫn đến có thể là do trong quá trình tiền huấn luyện mô hình không học được biểu diễn tương phản như trong

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
PhoBERT-large bi-encoder pha 1	87	87,33	88,33	88,33	**92,33**	**92,33**
PhoBERT-large Condenser bi-encoder pha 1	83	83,33	87,16	87,33	91,16	91,33
PhoBERT-large coCondenser bi-encoder pha 1	**88,33**	**88,67**	**89,67**	**89,67**	**92,33**	**92,33**

Bảng 4.4: Bảng kết quả độ chính xác Acc(%) và Recall(%) khi tinh chỉnh với backbone được tiền huấn luyện với các phương pháp khác nhau

kiến trúc coCondenser. Một nguyên nhân khác dẫn đến kết quả thấp cho thể do khi huấn luyện với dữ liệu pha 1 với việc khai thác nhiều đến thông tin về so khớp từ vựng nên việc học biểu diễn cho token CLS để tổng hợp toàn bộ thông tin của cả câu trở nên không hiệu quả.

4.6 So sánh kết quả khi tinh chỉnh các mô hình bi-encoder pha 1 với dữ liệu trước và sau khi chunking

Vì mô hình backbone PhoBERT-large chỉ hỗ trợ tối đa 256 token đầu vào, nên với các đoạn văn có kích thước vượt quá số token này, phần token dư thừa sẽ bị cắt bỏ (truncate) khi qua mô hình bi-encoder, dẫn đến mất mát thông tin. Để khắc phục vấn đề này,với mỗi cặp (câu truy vấn, đoạn văn) em đã chia đoạn văn này thành các đoạn (chunk) nhỏ hơn sao cho kích thước mỗi đoạn này đều nhỏ hơn kích thước tối đa mà backbone PhoBERT-large hỗ trợ. Mỗi chunk cùng với câu truy vấn sẽ tạo thành các mẫu dương để sử dụng cho quá trình tinh chỉnh. Bảng 4.5 em thực hiện so sánh kết quả khi tinh chỉnh các mô hình bi-encoder với dữ liệu pha 1 trước và sau khi chunking.

Từ kết quả của bảng so sánh, có thể thấy rằng các mô hình bi-encoder sau khi tinh chỉnh với dữ liệu đã được chunking đều cho kết quả tốt hơn đáng kể. Điều này có thể do thông tin cần thiết cho câu truy vấn nằm ở phần cuối của đoạn văn, vượt quá số lượng token tối đa mà mô hình bi-encoder hỗ trợ. Khi đó, việc học biểu diễn cho token CLS trở nên không hiệu quả. Phương pháp chunking giúp khắc phục vấn đề này bằng cách đảm bảo rằng tất cả các thông tin quan trọng đều được bao gồm trong quá trình tinh chỉnh.

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
PhoBERT-large bi-encoder pha 1	87	87,33	88,33	88,33	92,33	92,33
PhoBERT-large Condenser bi-encoder pha 1	83	83,33	87,16	87,33	91,16	91,33
PhoBERT-large coCondenser bi-encoder pha 1	88,33	88,67	89,67	89,67	92,33	92,33
PhoBERT-large bi-encoder pha 1 + chunking	88,67	89	91,83	92	**96,33**	**96,33**
PhoBERT-large Condenser bi-encoder pha 1 + chunking	87	87,33	90,5	90,66	95,16	95,33
PhoBERT-large coCondenser bi-encoder pha 1 + chunking	**92,05**	**92,33**	**95,33**	**95,33**	95,67	95,67

Bảng 4.5: Kết quả độ chính xác Acc(%) và Recall(%) trước và sau khi chunking dữ liệu các mô hình bi-encoder pha 1

4.7 So sánh kết quả khi tinh chỉnh mô hình bi-encoder pha 1 và pha 2 với dữ liệu sau khi chunking

Với các mô hình bi-encoder sau khi đã được tinh chỉnh ở pha một 1, em tiến hành thực hiện suy luận trên chính tập huấn luyện để tạo ra các mẫu âm khó và dùng các mẫu âm này làm dữ liệu tinh chỉnh ở pha 2. Kết quả so sánh giữa việc tinh chỉnh các mô hình bi-encoder với dữ liệu pha 1 và pha 2 được em trình bày trong bảng 4.6

Từ bảng so sánh, có thể thấy rằng các mô hình bi-encoder ở pha 2 có kết quả tốt hơn đáng kể so với các mô hình tương ứng ở pha 1. Điều này cho thấy tầm quan trọng của việc khai thác các mẫu âm trong pha 2. Ở pha 1, các mẫu âm được tạo ra bằng cách sử dụng BM25, một phương pháp tập trung vào việc so khớp từ vựng. Trong khi đó, ở pha 2, các mẫu âm được lấy từ kết quả suy luận của các mô hình bi-encoder pha 1 trên tập huấn luyện, và đây là những mẫu âm khó về mặt ngữ nghĩa. Việc sử dụng các mẫu âm khó này để tiếp tục tinh chỉnh giúp cải thiện hiệu quả của mô hình trong việc học biểu diễn.

4.8 So sánh kết quả khi tinh chỉnh mô hình M3-Embedding với các mô hình bi-encoder với cùng dữ liệu pha 1

Trong thử nghiệm này em tiến hành tinh chỉnh mô hình M3-Embedding trên tập dữ liệu pha 1 sau khi được chunking và khai phá bởi BM25. Sau khi tinh chỉnh

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
PhoBERT-large bi-encoder pha 1	88,67	89,00	91,83	92,00	96,33	96,33
PhoBERT-large Condenser bi-encoder pha 1	87,00	87,33	90,5	90,66	95,16	95,33
PhoBERT-large coCondenser bi-encoder pha 1	92,05	92,33	95,33	95,33	95,67	95,67
PhoBERT-large bi-encoder pha 2	89,33	89,67	93,83	94,00	97,16	97,33
PhoBERT-large Condenser bi-encoder pha 2	87,33	87,66	92,16	92,33	95,83	96,00
PhoBERT-large coCondenser bi-encoder pha 2	**95,66**	**96,00**	**96,67**	**96,67**	**97,67**	**97,67**

Bảng 4.6: Kết quả độ chính xác Acc(%) và Recall(%) của các mô hình bi-encoder pha 1 và pha 2

em có đánh giá kết quả của từng loại chức năng truy xuất bao gồm truy xuất dày đặc (Dense retrieval), truy xuất thưa thớt (Sparse retrieval) và truy xuất đa véc-tơ (Multi-vec retrieval). Bảng 4.7 so sánh kết quả từng loại chức năng truy xuất này với các mô hình bi-encoder pha 1.

Từ bảng so sánh có thể thấy được mô hình M3-Embedding đạt kết quả vượt trội hơn hẳn trên tất cả các tiêu chí đánh giá so với 3 mô hình bi-encoder chỉ riêng với mỗi chức năng truy xuất dày đặc (Ký hiệu trong bảng là Dense + chunking). Điều này đã chứng minh được sự ưu việt của phương pháp tự chắt lọc tri thức (self-knowledge distillation). Đạt kết quả cao nhất phải kể đến chức năng truy xuất đa véc-tơ (Ký hiệu trong bảng là Multi-vec + chunking), chức năng truy xuất này dựa vào sự tương tác chi tiết giữa các token của câu truy vấn và đoạn văn để tính sự tương đồng.

4.9 So sánh kết quả khi tinh chỉnh mô hình M3-Embedding với dữ liệu pha 1 và pha 2

Trong thử nghiệm này em thực hiện khai thác các mẫu âm khó pha 2. Cả 3 chức năng truy xuất của pha 1 sau khi qua bước ensemble sẽ được dùng để tạo ra các mẫu âm khó pha 2. Kết quả so sánh của các chức năng truy xuất của mô hình M3-embedding pha 1 và pha 2 được em trình bày trong bảng 4.8

Sau khi khai phá tiếp các mẫu âm khó cho việc tinh chỉnh mô hình M3-

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
Mô hình bi-encoder pha 1						
PhoBERT-large bi-encoder + chunking	88,67	89,00	91,83	92,00	96,33	96,33
PhoBERT-large Condenser bi-encoder + chunking	87,00	87,33	90,5	90,66	95,16	95,33
PhoBERT-large coCondenser bi-encoder + chunking	92,05	92,33	95,33	95,33	95,67	95,67
Mô hình M3-Embedding pha 1						
Dense + chunking	95,00	95,33	95,83	96,00	96,67	96,67
Sparse + chunking	88,38	89,33	92,33	93,00	94,5	94,5
Multi-vec + chunking	**96,33**	**96,33**	**97,00**	**97,00**	**97,33**	**97,33**

Bảng 4.7: Kết quả độ chính xác Acc(%) và Recall(%) của M3-Embedding so với các mô hình bi-encoder pha 1

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
Mô hình M3-Embedding pha 1						
Dense + chunking	95,00	95,33	95,83	96,00	96,67	96,67
Sparse + chunking	88,38	89,33	92,33	93,00	94,5	94,5
Multi-vec + chunking	**96,33**	**96,33**	97,00	97,00	97,33	97,33
Mô hình M3-Embedding pha 2						
Dense + chunking	96,16	**96,33**	97,33	97,33	97,67	97,67
Sparse + chunking	90.27	91,00	93,05	93,66	95,55	96,00
Multi-vec + chunking	**96,33**	**96,33**	**97,67**	**97,67**	**98,67**	**98,67**

Bảng 4.8: Kết quả độ chính xác Acc(%) và Recall(%) của mô hình M3-Embedding pha 1 và pha 2

Embedding ở pha 2 thì có thể thấy được kết quả trên từng chức năng truy xuất đều có sự cải thiện, trong đó cả 2 độ đo Recall@10 và Acc@10 đều đạt tới kết quả 98,67% .

4.10 So sánh kết quả với các mô hình baseline

Trong thử nghiệm này em sẽ so sánh kết quả của 3 mô hình bi-encoder cũng như là 3 chức năng truy xuất của mô hình M3-Embedding trong pha 2 với các mô hình baseline và BM25 mà em đã chọn. Bên cạnh đó em cũng sẽ bổ sung thêm kết quả khi ensemble mô hình BM25 với chức năng truy xuất của BM25 và kết quả khi ensemble 3 mô hình bi-encoder và 3 chức năng truy xuất của M3-Embedding. Toàn bộ kết quả trên được em trình bày trong bảng 4.9 .

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
BM25	73,61	74,33	79,38	79,67	87,5	88,00
vietnamese-sbert	44,38	45,00	51,5	52,00	58,27	59,00
vietnamese-bi-encoder	60,77	61,33	68,61	69,00	76,67	77,00
PhoBERT-large bi-encoder pha 2	89,33	89,67	93,83	94,00	97,16	97,33
PhoBERT-large Condenser bi-encoder pha 2	87,33	87,66	92,16	92,33	95,83	96,00
PhoBERT-large coCondenser bi-encoder pha 2	95,66	96,00	96,67	96,67	97,67	97,67
Dense M3-Embedding pha 2	96,16	96,33	97,33	97,33	97,67	97,67
Sparse M3-Embedding pha 2	90.27	91,00	93,05	93,66	95,55	96,00
Multi-vec M3-Embedding pha 2	96,33	96,33	97,67	97,67	98,67	98,67
Ensemble	**96,83**	**97,00**	**98,00**	**98,00**	**99,00**	**99,00**

Bảng 4.9: Kết quả độ chính xác Acc(%) và Recall(%) khi so sánh với các mô hình baseline

Từ bảng so sánh đầu tiên có thể dễ dàng thấy được cùng là truy xuất thưa thớt nhưng kết quả chức năng truy xuất thưa thớt (trong bảng là: Sparse M3-Embedding pha 2) có kết quả vượt trội hơn hẳn so với mô hình BM25. Điều này cho thấy được sự hiệu quả của việc sử dụng kĩ thuật tự chắt lọc tri thức (Self-Knowledge Distillation) trong việc tinh chỉnh mô hình M3-Embedding .

Cho kết quả thấp nhất trong tất cả các mô hình trên đó là mô hình baseline vietnamese-sbert. Nguyên nhân có thể do hàm mất mát Cosine Similarity của mô hình này không thực sự hiệu quả cho việc học không gian biểu diễn. Cho kết quả thấp thứ hai đó là mô hình baseline vietnamese-bi-encoder, có thể do mô hình này

chưa được tinh chỉnh trên tập dữ liệu thuộc miền thông tin của Đại học Bách Khoa Hà Nội, dẫn đến hiệu quả không cao.

Khi ensemble kết quả của BM25 với truy xuất thưa thớt của mô hình M3-Embedding thì kết quả tốt hơn so với kết quả riêng của từng mô hình. Điều này cũng tương tự khi em thực hiện ensemble kết quả 3 mô hình bi-encoder cùng với 3 chức năng truy xuất của M3-Embedding (trong bảng là: Ensemble) và kết quả này đã đạt được kết quả cao nhất trong bảng so sánh. Qua đây cũng có thể thấy được việc ensemble đã giúp các mô hình bổ sung các điểm mạnh cho nhau và làm cho kết quả tổng hợp của các mô hình trở nên tốt hơn .

4.11 Kết quả trên toàn bộ mô-đun truy xuất

Trong thử nghiệm này em so sánh kết quả truy xuất của ba mô hình bi-encoder pha 2, ba chức năng truy xuất của mô hình M3-Embedding pha 2, ensemble ba mô hình bi-encoder và ba chức năng truy xuất của mô hình M3-Embedding pha 2 , ensemble ba mô hình bi-encoder và ba chức năng truy xuất của mô hình M3-Embedding pha 2 kết hợp với kết quả của cross-encoder, ensemble ba mô hình bi-encoder và ba chức năng truy xuất của mô hình M3-Embedding pha 2 kết hợp với mô hình BM5 và cuối cùng là ensemble ba mô hình bi-encoder và ba chức năng truy xuất của mô hình M3-Embedding pha 2 và bổ sung thêm cả cross-encoder và BM25. Kết quả thử nghiệm được em trình bày trong bảng 4.10.

Từ bảng kết quả có thể thấy được việc thêm mô hình BM25 cũng như là mô hình cross-encoder đều giúp tăng kết quả truy xuất. Mô hình cross-encoder được huấn luyện trên các mẫu âm khó ở pha 3 , đây là các mẫu âm mà ba mô hình bi-encoder và mô hình M3-Embedding vẫn chưa học tốt. Cách huấn luyện này sẽ giúp cho mô hình cross-encoder học được tốt mối liên hệ giữa câu truy vấn và đoạn văn bản. Đồng thời việc thêm BM25 vào trong mô-đun, sẽ giúp kết quả cải thiện hơn trong những trường hợp nặng về so khớp từ vựng. Và kết quả tốt nhất trong bảng chính là khi bổ sung thêm đồng thời BM25 và cross-encoder. Đây cũng chính là kết quả cuối cùng em sử dụng cho mô-đun truy xuất trong hệ thống.

4.12 So sánh về thời gian truy xuất các mô hình trong mô-đun truy xuất

Trong thử nghiệm này em so sánh thời gian truy xuất trung bình trên 1 câu truy vấn của các mô hình bao gồm BM25, ba mô hình bi-encoder pha 2, ba chức năng truy xuất của mô hình M3-Embedding pha 2, sau đó là thời gian ensemble kết quả ba mô hình bi-encoder với ba chức năng truy xuất của M3-Embedding và cuối cùng là thời gian thử nghiệm khi bổ sung mô hình cross-encoder để xếp hạng lại dựa trên tốp 20 văn bản phù hợp được trích xuất ra từ kết quả ensemble ba mô

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
PhoBERT-large bi-encoder pha 2	89,33	89,67	93,83	94	97,16	97,33
PhoBERT-large Condenser bi-encoder pha 2	87,33	87,66	92,16	92,33	95,83	96
PhoBERT-large coCondenser bi-encoder pha 2	95,66	96	96,67	96,67	97,67	97,67
Dense M3-Embedding pha 2	96,16	96,33	97,33	97,33	97,67	97,67
Sparse M3-Embedding pha 2	90.27	91	93,05	93,66	95,55	96
Multi-vec M3-Embedding pha 2	96,33	96,33	97,67	97,67	98,67	98,67
Ensemble 3 mô hình bi-encoder và 3 loại truy xuất M3-Embedding pha 2	96,83	97	98	98	99	99
Ensemble 3 mô hình bi-encoder và 3 loại truy xuất M3-Embedding pha 2 + cross-encoder	97,00	97,00	**98,33**	**98,33**	99,00	99,00
Ensemble 3 mô hình bi-encoder và 3 loại truy xuất M3-Embedding pha 2 + BM25	96,83	97,00	98,00	98,00	99,00	99,00
Ensemble 3 mô hình bi-encoder và 3 loại truy xuất M3-Embedding pha 2 + BM25 + cross-encoder	**97,33**	**97,33**	**98,33**	**98,33**	**99,33**	**99,33**

Bảng 4.10: Kết quả độ chính xác Acc(%) và Recall(%) trên toàn bộ mô-đun truy xuất

hình bi-encoder với ba chức năng truy xuất của M3-Embedding. Kết quả của thử nghiệm này được em trình bày trong bảng 4.13.

Từ bảng so sánh có thể thấy mô hình BM25 có thời gian truy xuất nhanh nhất, chỉ mất 0,0086 giây. Điều này cho thấy BM25 có khả năng xử lý truy vấn nhanh chóng, phù hợp cho các hệ thống yêu cầu tốc độ cao.

Các mô hình bi-encoder có thời gian truy xuất dao động từ 0,03 đến 0,045 giây.

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
PhoBERT-large bi-encoder pha 2	89,33	89,67	93,83	94	97,16	97,33
PhoBERT-large Condenser bi-encoder pha 2	87,33	87,66	92,16	92,33	95,83	96
PhoBERT-large coCondenser bi-encoder pha 2	95,66	96	96,67	96,67	97,67	97,67
PhoBERT-large bi-encoder pha 2 + BM25	90,66	91	94,83	95	97,16	97,33
PhoBERT-large Condenser bi-encoder pha 2 + BM25	89,16	90,00	92,16	92,33	96,67	96,67
PhoBERT-large coCondenser bi-encoder pha 2 + BM25	96,00	96.00	97,33	97,33	98,33	98,33
PhoBERT-large bi-encoder pha 2 + re-rank	95,83	96,00	97,00	97,00	98,00	98,00
PhoBERT-large Condenser bi-encoder pha 2 + re-rank	96,33	96,66	97,16	97,33	98,16	98,33
PhoBERT-large coCondenser bi-encoder pha 2 + re-rank	96,33	96,66	97,16	97,33	98,33	98,33

Bảng 4.11: Kết quả độ chính xác Acc(%) và Recall(%) trên toàn bộ mô-đun truy xuất

Điều này cho thấy các mô hình này có hiệu suất tương đối ổn định và nhanh chóng. Tiếp đến có thể thấy trong ba chức năng truy xuất của M3-Embedding thì truy xuất thưa thớt (ký hiệu trong bảng là Multi-vec M3-Embedding pha 2) có thời gian truy xuất lâu nhất, điều này có thể giải thích bởi chức năng truy xuất này tính toán điểm của mỗi token trong câu quy vấn với tất cả token của tài liệu.

Khi ensemble kết hợp ba mô hình bi-encoder và ba chức năng truy xuất M3-Embedding có thời gian truy xuất là 0,243 giây và việc kết hợp này vẫn đảm bảo được thời gian xử lý tương đối nhanh. Tiếp đó khi bổ sung thêm mô hình cross-encoder để thực hiện xếp hạng lại thì thời gian truy xuất đã tăng lên đáng kể từ 0,243 tới 4,883 giây.

Với thời gian truy xuất của các mô hình em nêu ở trong bảng so sánh thì đều áp ứng được về mặt thời gian để triển khai trong thực tế. Tuy nhiên để có thể giảm

Mô hình	k=3		k=5		k=10	
	Recall	Acc	Recall	Acc	Recall	Acc
PhoBERT-large bi-encoder pha 2	89,33	89,67	93,83	94	97,16	97,33
PhoBERT-large Condenser bi-encoder pha 2	87,33	87,66	92,16	92,33	95,83	96
PhoBERT-large coCondenser bi-encoder pha 2	95,66	96	96,67	96,67	97,67	97,67
Dense M3-Embedding pha 2	96,16	96,33	97,33	97,33	97,67	97,67
Sparse M3-Embedding pha 2	90.27	91	93,05	93,66	95,55	96
Multi-vec M3-Embedding pha 2	96,33	96,33	97,67	97,67	98,67	98,67
Ensemble	96,83	97	98	98	99	99
Ensemble + re-rank	97,00	97,00	**98,33**	**98,33**	99,00	99,00
Ensemble + BM25	97,00	97,00	98,00	**98,33**	99,00	99,00
Ensemble + BM25 + re-rank	**97,33**	**97,33**	**98,33**	**98,33**	**99,33**	**99,33**

Bảng 4.12: Kết quả độ chính xác Acc(%) và Recall(%) trên toàn bộ mô-đun truy xuất

Mô hình	Thời gian
BM25	0,0086
PhoBERT-large bi-encoder pha 2	0,045
PhoBERT-large Condenser bi-encoder pha 2	0,03
PhoBERT-large coCondenser bi-encoder pha 2	0,036
Dense M3-Embedding pha 2	0,044
Sparse M3-Embedding pha 2	0,054
Multi-vec M3-Embedding pha 2	0,121
Ensemble 3 mô hình bi-encoder và 3 chức năng truy xuất M3-Embedding pha 2	0,243
Ensemble 3 mô hình bi-encoder và 3 chức năng truy xuất M3-Embedding pha 2 + cross-encoder	4,883

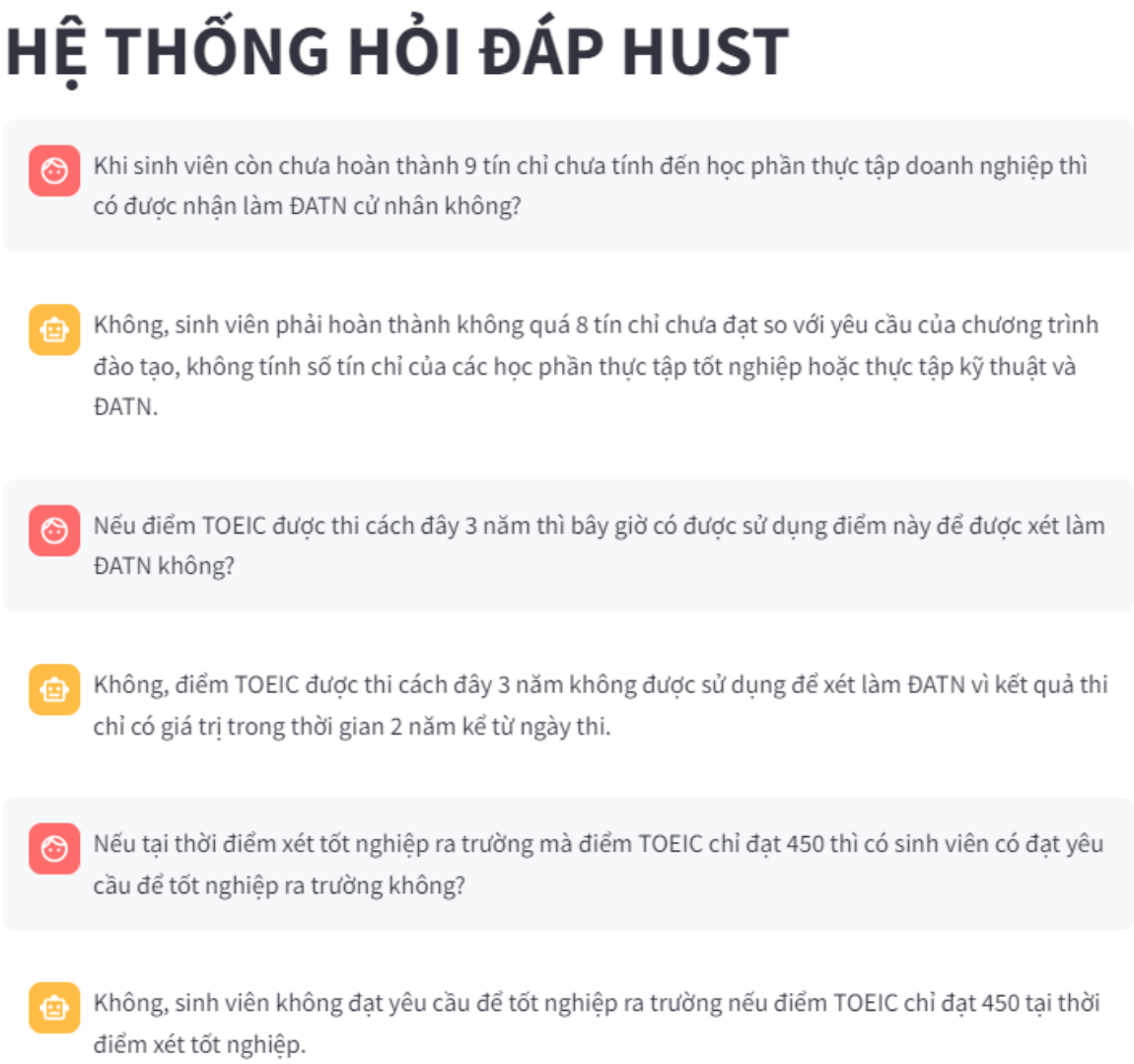
Bảng 4.13: So sánh thời gian truy xuất (đơn vị giây) của các mô hình trong mô đun truy xuất

thời gian truy vấn hơn nữa thì vẫn cần phải có thêm đầu tư về phần cứng cũng như là cơ sở hạ tầng để giúp các mô hình này tính có thể tính toán song song.

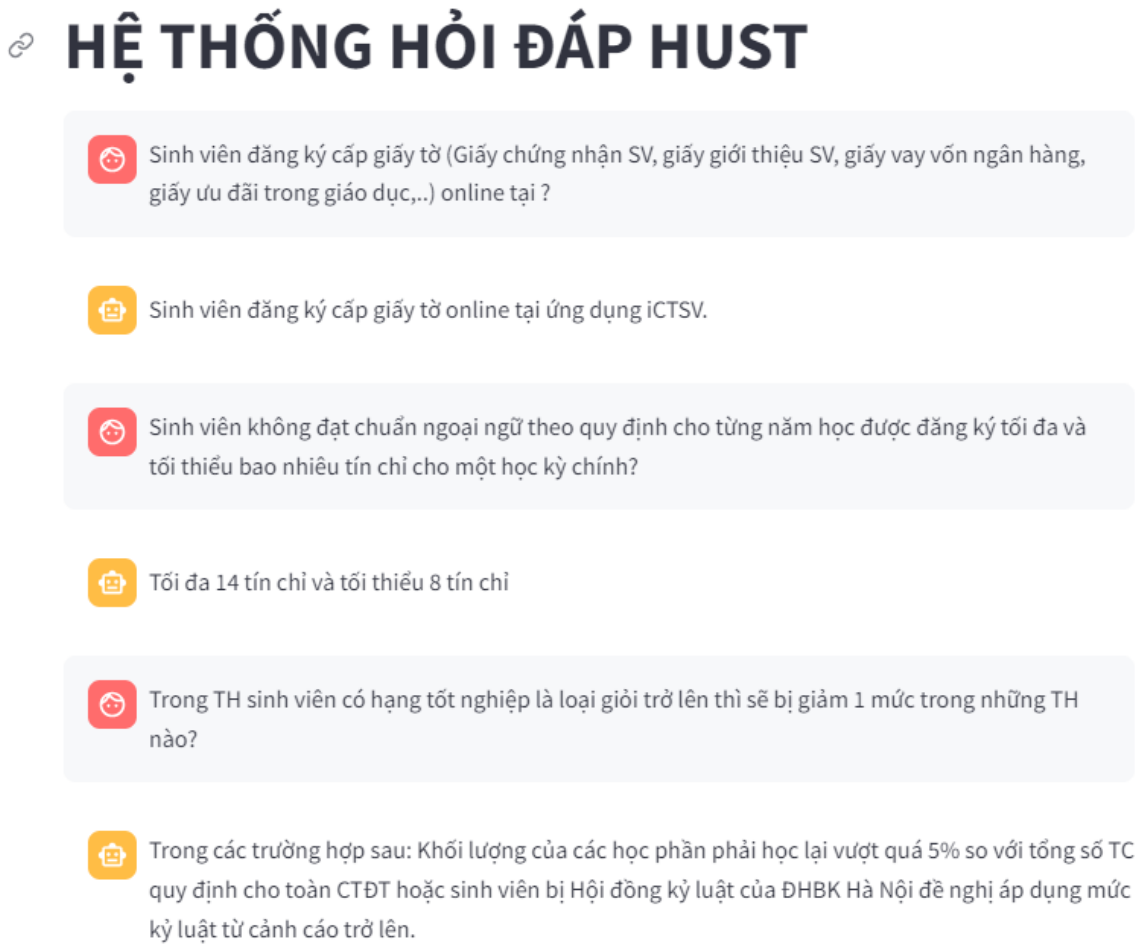
4.13 Kết quả kiểm thử toàn hệ thống

Toàn bộ hệ thống của em được triển khai sử dụng công cụ Streamlit, một thư viện mã nguồn mở cho Python, giúp tạo ra các ứng dụng web tương tác một cách nhanh chóng và dễ dàng. Để minh họa và kiểm chứng tính hiệu quả của hệ thống hỏi đáp thì dưới đây là các hình ảnh khi em tương tác trực tiếp với hệ thống của mình.

Hình 4.1: Minh họa khi tương tác với hệ thống - phần 1



Hình 4.2: Minh họa khi tương tác với hệ thống - phần 2



Hình 4.3: Minh họa khi tương tác với hệ thống - phần 3

Hình 4.4: Minh họa khi tương tác với hệ thống - phần 4

CHƯƠNG 5. KẾT LUẬN

Trong chương này em sẽ đưa ra kết luận chung cho toàn bộ đồ án bao gồm các vấn đề mà em đã giải quyết được cũng như các vấn đề vẫn chưa được giải quyết ở phần 5.1 và sau đó em sẽ đưa ra các hướng phát triển tiếp theo cho đồ án của mình ở phần 5.2.

5.1 Kết luận

Qua đồ án em đã xây dựng được một hệ thống hỏi đáp hoàn chỉnh cho các bạn sinh viên Đại học Bách Khoa Hà Nội, hệ thống cho câu trả lời nhanh và chính xác và có thể đáp ứng trong việc hỗ trợ cho các bạn sinh viên trong thực tế.

Đồ án của em đã khắc phục được những yếu điểm trong việc xây dựng hệ thống hỏi đáp, cũng như giải quyết các vấn đề tồn đọng trong từng mô-đun của hệ thống hỏi đáp từ các nghiên cứu trước đây. Chi tiết các vấn đề mà đồ án của em đã giải quyết bao gồm có:

1. Áp dụng kỹ thuật chunking dữ liệu để giúp nâng cao việc đào tạo mô hình bi-encoder và cross-encoder.
2. Việc kết hợp giữa hai thành phần retriever và re-ranker trong mô-đun truy xuất để tận dụng được thế mạnh về tốc độ của retriever cũng như độ chính xác cao của re-ranker.
3. Sử dụng các kiến trúc Transformer chuyên để tiền huấn luyện hiệu quả cho tác vụ truy xuất thông tin đó là Condenser và coCondenser.
4. Thực hiện khai phá các mẫu âm khó qua nhiều pha giúp cho việc học biểu diễn tương phản trở nên hiệu quả hơn.
5. Thông qua kĩ thuật tự chắt lọc tri thức (Self-Knowledge Distillation) giúp huấn luyện đồng thời 3 chức truy xuất bao gồm truy xuất dày đặc, truy xuất thưa thớt và truy xuất đa véc-tơ nhằm nâng cao chất lượng của từng chức năng truy xuất này.
6. Kết hợp kết quả của nhiều mô hình đơn lẻ (ensemble) để tận dụng sức mạnh của từng mô hình.
7. Việc kết hợp thêm mô hình BM25 vào mô-đun truy xuất đã giúp tăng hiệu quả thông qua việc so khớp từ vựng.
8. Sử dụng các kỹ thuật prompting trên mô hình ngôn ngữ lớn mã nguồn mở đã giúp cho việc sinh câu trả lời tự nhiên và chính xác hơn và tránh được chi phí cao cho việc huấn luyện các mô hình trả lời câu hỏi.

Bên cạnh đó thì vẫn còn những vấn đề mà đồ án vẫn cần phải cải thiện thêm bao gồm:

1. Do thời gian thu thập dữ liệu có giới hạn và có nhiều thông tin được phân bố ở nhiều trang khác nhau nên tập dữ liệu của em đã tổng hợp chưa đủ để bao quát toàn bộ nội dung thông tin cũng như là các quy định, quy chế của Đại học Bách Khoa Hà Nội. Bên cạnh đó thì các nội dung thông tin này vẫn có thể sẽ được sửa đổi theo thời gian, vậy nên hệ thống hỏi đáp cũng cần phải thiết kế cơ sở dữ liệu sao cho có thể đáp ứng được sự thay đổi về dữ liệu này.
2. Hiện tại hệ thống vẫn chỉ đang dừng ở mức độ cá nhân và chưa được em đựa vào thực tế sử dụng, vậy nên các bạn sinh viên vẫn chưa được trải nghiệm để có được đánh giá khách quan nhất về hệ thống.

5.2 Hướng phát triển tiếp theo

Trong hướng phát triển tiếp theo em sẽ tập trung vào các vấn đề mà đồ án của em cần phải cải thiện thêm bao gồm:

1. Có thể thu thập đầy đủ nhất toàn bộ dữ liệu thông tin các quy định, quy chế của Đại học Bách Khoa Hà Nội. Bên cạnh đó em cũng sẽ tìm hiểu các cách để thiết kế cơ sơ dữ liệu sao cho nó luôn luôn sẵn sàng cập nhật khi có một thông tin văn bản nào đó được sửa đổi.
2. Triển khai hệ thống này vào thực tế để tất cả các bạn sinh viên đều có thể truy cập và sử dụng.
3. Tìm hiểu và thử nghiệm thêm các mô hình và kỹ thuật mới cho cả hai mô-đun truy xuất và sinh câu trả lời để tăng độ chính xác cũng như giảm được thời gian chạy của hệ thống.

Lời cuối cùng cho đồ án này, em xin được gửi lời cảm ơn chân thành đến thầy Ngô Văn Linh, cảm ơn thầy vì đã hướng dẫn em vô cùng nhiệt tình và cho em nhiều động lực để hoàn thành đồ án tốt nghiệp này.

TÀI LIỆU THAM KHẢO

[1] S. E. Robertson, H. Zaragoza **and** M. J. Taylor, "Simple BM25 extension to multiple weighted fields," **in***Proceedings of the 2004 ACM CIKM International Conference on Information and Knowledge Management, Washington, DC, USA, November 8-13, 2004* D. A. Grossman, L. Gravano, C. Zhai, O. Herzog **and** D. A. Evans, **editors**, ACM, 2004, **pages** 42–49.

[2] G. Salton **and** C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manag.*, **jourvol** 24, **number** 5, **pages** 513–523, 1988.

[3] J. Devlin, M. Chang, K. Lee **and** K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," J. Burstein, C. Doran **and** T. Solorio, **editors**, Association for Computational Linguistics, 2019, **pages** 4171–4186.

[4] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado **and** J. Dean, "Distributed representations of words and phrases and their compositionality," **in***Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States* C. J. C. Burges, L. Bottou, Z. Ghahramani **and** K. Q. Weinberger, **editors**, 2013, **pages** 3111–3119.

[5] J. Pennington, R. Socher **and** C. D. Manning, "Glove: Global vectors for word representation," **in***Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL* A. Moschitti, B. Pang **and** W. Daelemans, **editors**, ACL, 2014, **pages** 1532–1543.

[6] H. Palangi, L. Deng, Y. Shen **andothers**, "Deep sentence embedding using long short-term memory networks: Analysis and application to information retrieval," *IEEE ACM Trans. Audio Speech Lang. Process.*, **jourvol** 24, **number** 4, **pages** 694–707, 2016.

[7] Y. Shen, X. He, J. Gao, L. Deng **and** G. Mesnil, "A latent semantic model with convolutional-pooling structure for information retrieval," **in***Proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management, CIKM 2014, Shanghai, China, November 3-7, 2014* J. Li, X. S. Wang, M. N. Garofalakis, I. Soboroff, T. Suel **and** M. Wang, **editors**, ACM, 2014, **pages** 101–110.

[8] A. Vaswani, N. Shazeer, N. Parmar **andothers**, "Attention is all you need," *CoRR*, **jourvol** abs/1706.03762, 2017. arXiv: `1706.03762`.

[9] Z. A. Yilmaz, S. Wang, W. Yang, H. Zhang **and** J. Lin, "Applying BERT to document retrieval with birch," **in***Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019 - System Demonstrations* S. Padó **and** R. Huang, **editors**, Association for Computational Linguistics, 2019, **pages** 19–24.

[10] N. Reimers **and** I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," **in***Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019* K. Inui, J. Jiang, V. Ng **and** X. Wan, **editors**, Association for Computational Linguistics, 2019, **pages** 3980–3990.

[11] L. Gao **and** J. Callan, "Condenser: A pre-training architecture for dense retrieval," **in***Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021* M. Moens, X. Huang, L. Specia **and** S. W. Yih, **editors**, Association for Computational Linguistics, 2021, **pages** 981–993.

[12] L. Gao **and** J. Callan, "Unsupervised corpus aware language model pre-training for dense passage retrieval," *CoRR*, **jourvol** abs/2108.05540, 2021. arXiv: `2108.05540`.

[13] K. M. Hermann, T. Kociský, E. Grefenstette **andothers**, "Teaching machines to read and comprehend," **in***Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada* C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama **and** R. Garnett, **editors**, 2015, **pages** 1693–1701. **url**: `https://proceedings.neurips.cc/paper/2015/hash/afdec7005cc9f14302cd0474fd0f3c96-Abstract.html`.

[14] D. Chen, J. Bolton **and** C. D. Manning, "A thorough examination of the cnn/daily mail reading comprehension task," **in***Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers* The Association for Computer Linguistics, 2016.

[15] M. J. Seo, A. Kembhavi, A. Farhadi **and** H. Hajishirzi, "Bidirectional attention flow for machine comprehension," **in***5th International Conference on Learning*

Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings OpenReview.net, 2017.

[16] M. Re **and** G. Valentini, "Ensemble methods: A review," **injanuary** 2012 **pages** 563–594.

[17] O. Khattab **and** M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over BERT," *CoRR*, **jourvol** abs/2004.12832, 2020. arXiv: 2004.12832.

[18] Y. Qu, Y. Ding, J. Liu **andothers**, "Rocketqa: An optimized training approach to dense passage retrieval for open-domain question answering," *CoRR*, **jourvol** abs/2010.0 2020. arXiv: 2010.08191.

[19] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian **and** Z. Liu, "BGE m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *CoRR*, **jourvol** abs/2402.03216, 2024. arXiv: 2402.03216.

[20] A. Conneau, K. Khandelwal, N. Goyal **andothers**, "Unsupervised cross-lingual representation learning at scale," **in***Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* Online: Association for Computational Linguistics, **july** 2020, **pages** 8440–8451.

[21] S. Xiao, Z. Liu, Y. Shao **and** Z. Cao, "RetroMAE: Pre-training retrieval-oriented language models via masked auto-encoder," **in***Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing* Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, **december** 2022, **pages** 538–548.

[22] J. Wei, M. Bosma, V. Y. Zhao **andothers**, *Finetuned language models are zero-shot learners*, 2022. arXiv: 2109.01652.

[23] T. B. Brown, B. Mann, N. Ryder **andothers**, *Language models are few-shot learners*, 2020. arXiv: 2005.14165.

[24] D. Q. Nguyen **and** A. T. Nguyen, "PhoBERT: Pre-trained language models for Vietnamese," **in***Findings of the Association for Computational Linguistics: EMNLP 2020* 2020, **pages** 1037–1042.

[25] Y. Liu, M. Ott, N. Goyal **andothers**, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.